

Benutzerhandbuch

Akkus testen mit ZKE-EBC-Testern – eigenständig, mit Aufzeichnung auf SD-Karte, bedient über den Browser

Firmware	1.4.72 (2. November 2026)
Hardware	EBCTester-Platine, Revision 1 (Schaltplan und Layout vom 4. November 2026)
Unterstützte Tester	ZKE EBC-A20, EBC-A40L, EBC-A50L
Dokument	Ausgabe November 2026, Deutsch
Projekt	github.com/Gemelon/EBCTester

© 2026 Thomas Stoll. Software und Dokumentation stehen unter der Apache License 2.0. EBCTester ist ein unabhängiges Hobbyprojekt und kein Produkt von ZKETECH; der Hersteller unterstützt es nicht.

Inhalt

Teil I – Bedienungsanleitung . . . 4

1 **Einführung** . . . 5
- 1.1 Was EBCTester macht . . . 5
- 1.2 Die wichtigsten Funktionen . . . 5
- 1.3 Unterstützte Tester . . . 6
- 1.4 Was Sie brauchen . . . 6
- 1.5 Sicherheitshinweise . . . 6

2 **Inbetriebnahme** . . . 8
- 2.1 Hardware anschließen . . . 8
- 2.2 Erster Start und Anmeldung . . . 9
- 2.3 Mit dem Heim-WLAN verbinden (optional) . . . 10
- 2.4 Testermodell auswählen . . . 10
- 2.5 LEDs auf der Platine . . . 11
- 2.6 Die Weboberfläche im Überblick . . . 11

3 **Bereich Batterietester** . . . 13
- 3.1 Messwerte . . . 13
- 3.2 Batterie . . . 14
- 3.3 Manuelle Bedienung . . . 16
- 3.4 Automatische Bedienung (Testpläne) . . . 17
- 3.5 Innenwiderstand (DCIR) . . . 19

4 **Bereich Verwaltung** . . . 23
- 4.1 Geräteinformation . . . 23
- 4.2 Testlaufverwaltung . . . 24
- 4.3 SD-Karte . . . 25
- 4.4 Updates . . . 26
- 4.5 Einstellungen . . . 27
- 4.6 Debug . . . 32

5 **Typische Abläufe** . . . 33
- 5.1 Kapazität eines Akkus messen . . . 33
- 5.2 Innenwiderstand bei 50 % Ladezustand messen . . . 33
- 5.3 Daten sichern . . . 33

6 **Fehlersuche** . . . 34

Teil II – Technische Referenz . . . 36

7 **Systemarchitektur** . . . 37
- 7.1 Aufbau der Firmware . . . 37
- 7.2 Datenfluss . . . 37

8 **Hardware** . . . 38
- 8.1 Blockschaltbild . . . 38
- 8.2 Schaltungsbeschreibung . . . 38
- 8.3 Anschlüsse, Lötbrücke und Messpunkte . . . 40
- 8.4 Stückliste . . . 43
- 8.5 Leiterplatte . . . 45

9 **Firmware: Installation und Konfiguration** . . . 46
- 9.1 Release installieren (ohne Entwicklungsumgebung) . . . 46
- 9.2 Aus dem Quellcode bauen . . . 46
- 9.3 Firmware-Voreinstellungen und Platinenrevision 1 . . . 47
- 9.4 Partitionstabelle . . . 47

9.5 Status-LED in Firmware 1.4.72 . . . 48

10 Kommunikation mit dem Tester . . . 49

10.1 Serielle Schnittstelle . . . 49
10.2 Modellerkennung . . . 49
10.3 Betriebsarten . . . 49

11 Datenspeicherung . . . 50

11.1 SQLite-Datenbank auf der SD-Karte . . . 50
11.2 Schemaversionen . . . 50
11.3 MariaDB-Sicherung . . . 50

12 Schnittstellen . . . 52

12.1 HTTP-API . . . 52
12.2 Home Assistant MQTT . . . 52

13 Firmware-Updates und Veröffentlichung . . . 54

13.1 Ablauf eines Updates . . . 54
13.2 Format des Manifests . . . 54
13.3 Eigene Builds veröffentlichen . . . 54

14 Sicherheit und Einschränkungen . . . 55

14.1 Sicherheit . . . 55
14.2 Bekannte Einschränkungen (Firmware 1.4.72) . . . 55
14.3 Lizenz . . . 55

Anhang A – Sprachdateien . . . 56

Anhang B – Glossar . . . 57

Teil I – Bedienungsanleitung

Dieser Teil beschreibt Einrichtung und täglichen Gebrauch von EBCTester: Tester anschließen, anmelden, Batterien erfassen, manuelle Tests, Testpläne und Innenwiderstandsmessungen durchführen sowie die aufgezeichneten Daten auswerten und verwalten. Programmier- oder Elektronikkenntnisse sind dafür nicht nötig.

1 Einführung

1.1 Was EBCTester macht

Die EBC-Batterietester von ZKE (ZKETECH) sind beliebte, preiswerte Geräte, um Akkus mit Konstantstrom, Konstantleistung oder Konstantspannung zu laden und zu entladen und ihre Kapazität zu messen. Normalerweise werden sie mit der Windows-Software des Herstellers gesteuert. Dafür muss während des ganzen Tests ein PC angeschlossen bleiben – oft tagelang.

EBCTester ersetzt diesen PC. Eine kleine Platine mit einem ESP32-S3-Mikrocontroller wird an die serielle Schnittstelle des Testers angeschlossen. Sie steuert den Tester, führt komplette Testpläne selbstständig aus und speichert jeden Messwert in einer Datenbank auf einer microSD-Karte. Bedient wird sie mit einem beliebigen Webbrowser – vom Handy, Tablet oder PC aus –, entweder über das heimische WLAN oder über den eigenen WLAN-Zugangspunkt des Geräts. Den Browser können Sie jederzeit schließen: Der Test läuft weiter und wird weiter aufgezeichnet.

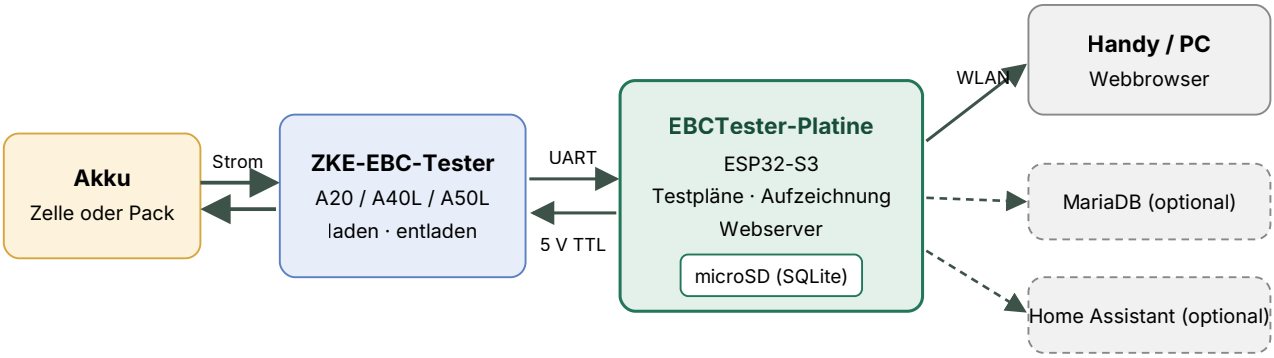


Abbildung 1: Systemübersicht

1.2 Die wichtigsten Funktionen

- **Kein PC während des Tests.** Manuelle Vorgänge und automatische Testpläne laufen auf dem ESP32-S3.
- **Aufzeichnung ohne Internet.** Alle Messwerte landen in einer SQLite-Datenbank auf einer FAT32-microSD-Karte. Für die Aufzeichnung ist eine betriebsbereite SD-Karte nötig, keine Internetverbindung.
- **Bedienung im Browser.** Diagramme, Batteriedaten, Teststeuerung und Verwaltung. Alles – auch die Diagrammbibliothek – liefert das Gerät selbst aus; weder Cloud-Dienst noch Internetzugang sind nötig.
- **Testpläne** mit bis zu 10 Schritten: Entladen mit Konstantstrom oder Konstantleistung, Laden mit CC/CV, Warten und Wiederholen.
- **Innenwiderstand (DCIR)** per Lastpulstest mit drei getrennten Ergebnissen (standardmäßig nach 1, 5 und 10 Sekunden), wahlweise mit automatischer Vorbereitung auf einen Ziel-Ladezustand.
- **Testlaufverwaltung:** aufgezeichnete Testläufe suchen, filtern, umbenennen und löschen.
- **Optional:** Sicherung auf einen MariaDB-Server (z. B. auf einem NAS), Einbindung in Home Assistant über MQTT und ein Live-Debug-Protokoll.
- **Firmware-Updates von Hand** über WLAN: Neue Versionen werden angezeigt, aber erst nach Ihrer Bestätigung installiert.
- **Sprachen:** Englisch und Deutsch eingebaut, weitere Sprachen lassen sich von der SD-Karte laden.

1.3 Unterstützte Tester

Die Firmware erkennt eine Reihe von ZKE-Modellen an der Kennung in ihren Statusmeldungen. Steuern lassen sich nur die als „freigegeben“ gekennzeichneten Modelle; alle anderen werden nur angezeigt, bis ihr Betrieb bestätigt ist.

Modell	Max. Laden	Max. Entladen	Max. Leistung	Status
EBC-A20	18 V / 5 A	30 V / 20 A	85 W	freigegeben
EBC-A40L	5 V / 40 A	5 V / 40 A	200 W	freigegeben
EBC-A50L	5 V / 50 A	5 V / 50 A	250 W	freigegeben
EBC-A05(+), EBC-A10(+/H), EBC-A20+, EBD-M10, EBD-A10H, EBD-M03(+), EBD-M05, EBD-A100(L), EBC-A100L	wird erkannt			nur Anzeige

Diese Grenzen gelten für jeden manuellen Befehl, jeden Planschritt und jeden Parameter der Innenwiderstandsmessung. A40L und A50L sind Niederspannungstester (max. 5 V): Verwenden Sie sie nur mit einzelnen Lithiumzellen und passenden Einstellungen. EBCTester kennt keine Auswahl der Zellchemie – die eingegebenen Werte müssen zu Ihrem Akku passen.

> **Hinweis**
> Wenn Sie eines der nur angezeigten Modelle besitzen und es mit EBCTester testen können, teilen Sie Ihre Ergebnisse bitte über das GitHub-Projekt mit, damit das Modell freigegeben werden kann.

1.4 Was Sie brauchen

- Eine EBCTester-Platine (oder einen kompatiblen ESP32-S3-Aufbau mit 16 MB Flash, siehe Teil II) mit installierter Firmware.
- Einen unterstützten ZKE-Tester und ein Kabel vom 6-poligen Anschluss J4 der Platine zur seriellen (TTL-)Schnittstelle des Testers.
- Stromversorgung: das 19-V-Netzteil des EBC-A20 (über J4) oder ein USB-C-Netzteil.
- Eine 2,4-GHz-Antenne (das Modul ESP32-S3-WROOM-1U hat keine eingebaute Antenne).
- Eine microSD-Karte im Format **FAT32** (exFAT wird nicht unterstützt); ideal sind 4–32 GB.
- Eine Knopfzelle CR2032 für die Echtzeituhr (optional, siehe Teil II).
- Ein Handy, Tablet oder einen Computer mit aktuellem Webbrowser.

1.5 Sicherheitshinweise

> **Warnung**
> Akkus speichern viel Energie. Falsche Einstellungen, beschädigte Zellen oder schlechte Verbindungen können zu Überhitzung, Brand oder Explosion führen. Vor allem Lithiumakkus dürfen nie über ihre Ladeschlussspannung geladen oder unter ihre Entladeschlussspannung entladen werden.

- Geben Sie Ladespannungen, Ströme und Abschaltwerte laut Datenblatt des Akkus ein. EBCTester prüft nur die Grenzen des *Testers*, nicht die Ihres Akkus.
- Die Software ersetzt **keine** Schutzbeschaltung (BMS, Sicherung, Temperaturüberwachung). Lange Tests sind nicht automatisch sicher, wenn niemand dabei ist.

- Bricht die serielle Verbindung ab, kann EBCTester nicht garantieren, dass der Tester stoppt. Prüfen Sie den Tester nach jedem Verbindungsfehler vor Ort.
- Verwenden Sie Kabel und Steckverbinder, die für den Prüfstrom ausgelegt sind, und stellen Sie den Aufbau auf eine nicht brennbare Unterlage.
- Die Weboberfläche nutzt unverschlüsseltes HTTP. Verwenden Sie sie nur in Ihrem eigenen, vertrauenswürdigen Netz oder über den geschützten Zugangspunkt des Geräts – machen Sie sie nie aus dem Internet erreichbar.

2 Inbetriebnahme

2.1 Hardware anschließen

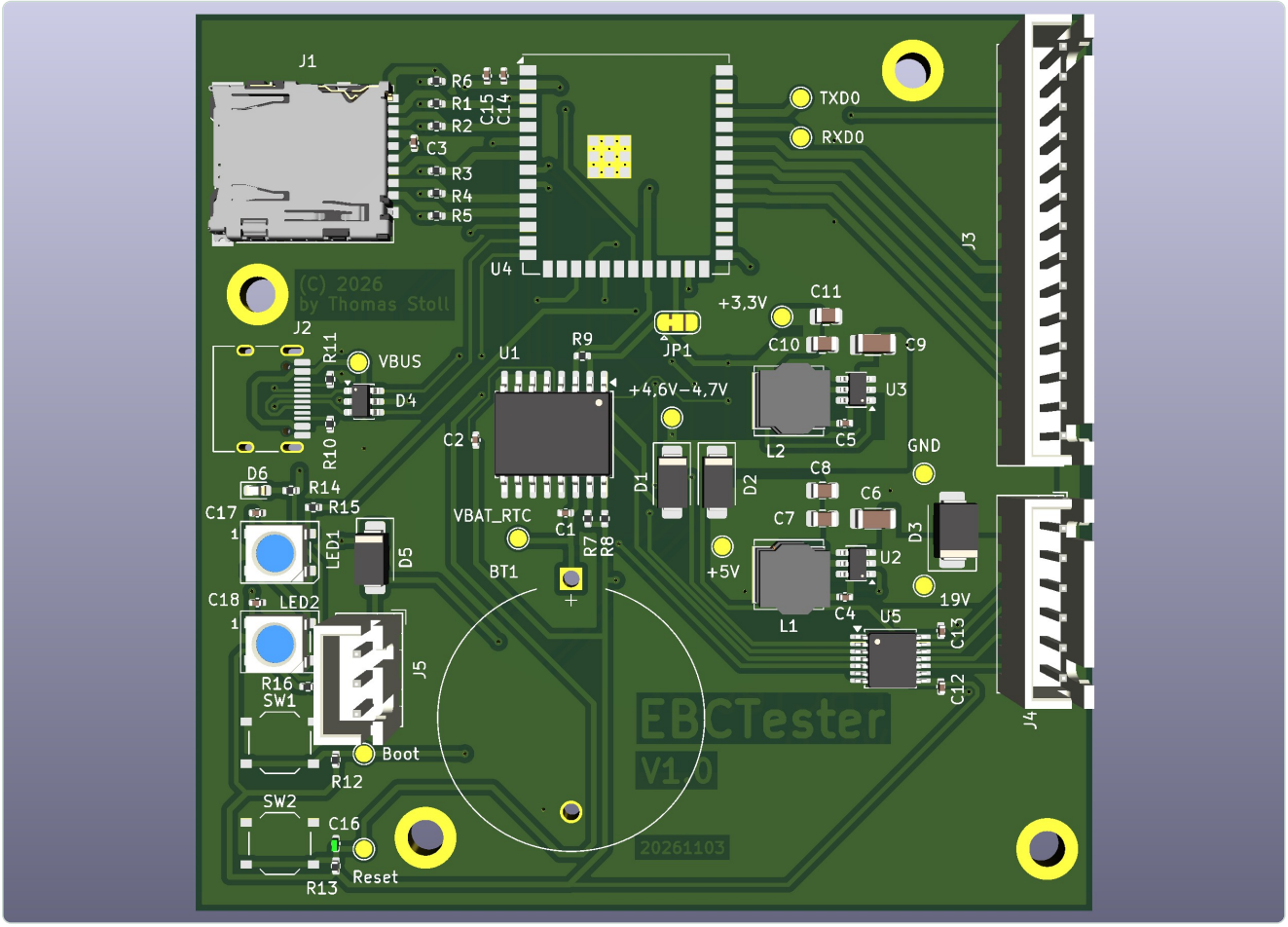


Abbildung 2: EBCTester-Platine, Revision 1 (Darstellung aus den Fertigungsdaten)

Die Anschlüsse liegen am Platinenrand: J4 für den Tester unten am rechten Rand, darüber der Erweiterungsstecker J3, oben links der microSD-Halter J1 und am linken Rand die USB-C-Buchse J2. LED1, LED2 und die Versorgungs-LED D6 sitzen links, die Taster BOOT und RESET in der linken unteren Ecke und der Halter für die CR2032 in der Mitte. Die Antennenbuchse (u.FL) befindet sich auf dem ESP32-Modul U4 oben auf der Platine.

1. **Antenne.** SMA-Buchse des Antennenkabels im Gehäuse befestigen, den kleinen u.FL-Stecker auf die Buchse des ESP32-Moduls stecken und die Antenne aufschrauben. Ohne Antenne funktioniert das WLAN nicht.
2. **microSD-Karte.** Eine FAT32-formatierte Karte einstecken. Die Karte wird nie automatisch formatiert; beim ersten Gebrauch wird darauf eine neue, leere Datenbank angelegt.
3. **Tester-Verbindung.** J4 mit der seriellen Schnittstelle des Testers verbinden (Belegung siehe Abschnitt 8.3). Die TX-Leitung der Platine geht zur RX-Leitung des Testers und umgekehrt. Beim EBC-A20 kann die Platine über J4 auch aus dem 19-V-Netzteil des Testers versorgt werden.
4. **Stromversorgung.** Die Platine entweder über J4 (19 V) oder über USB-C versorgen. Beides darf gleichzeitig angeschlossen sein.

> **Achtung**
> Die Pinbelegung des Kabels auf der Testerseite unterscheidet sich je nach Modell (der EBC-A40L hat einen anderen Stecker, und dort sollen TX/RX gegenüber dem A20-Kabel vertauscht sein). Prüfen Sie die Belegung vor dem Anschließen mit einem Multimeter. A40L und A50L werden direkt am Stromnetz betrieben und liefern keine 19 V – versorgen Sie die Platine dann über USB-C.

2.2 Erster Start und Anmeldung

Beim ersten Start öffnet das Gerät einen eigenen WLAN-Zugangspunkt (Access Point, AP) mit dem Namen `BatteryTester-XXXXXX`; *XXXXXX* ist für jedes Gerät anders. EBCTester verwendet zwei getrennte Passwörter:

Passwort	Wofür	Anfangswert
AP-Passwort	Verbindung mit dem WLAN `BatteryTester-XXXXXX`	Beim Erstellen der Firmware festgelegt; sonst einmalig zufällig erzeugt und im Startprotokoll ausgegeben
Anmeldepasswort	Anmeldung an der Weboberfläche (Benutzer `admin`) und Bestätigung jeder Einstellungsänderung	`ebctester`

Ein zufällig erzeugtes AP-Passwort lesen Sie so ab: Platine per USB-C mit einem PC verbinden, ein serielles Terminal öffnen (z. B. ESP-IDF-Monitor, PuTTY oder den seriellen Monitor der Arduino-IDE) und `RESET` drücken. Notieren Sie das Passwort – es bleibt auch nach Neustarts gleich.

1. Handy oder PC mit dem WLAN `BatteryTester-XXXXXX` verbinden und dabei das AP-Passwort eingeben.
2. Im Browser `http://192.168.4.1` öffnen.
3. Mit dem Benutzernamen `admin` und dem Anmeldepasswort anmelden – bei einem neuen Gerät `ebctester`. Auf der Anmeldeseite heißt dieses Feld **Admin-Passwort**.
4. Das Anmeldepasswort gleich unter **Verwaltung → Einstellungen → Anmeldeeinstellungen** ändern (siehe Anmeldeeinstellungen).

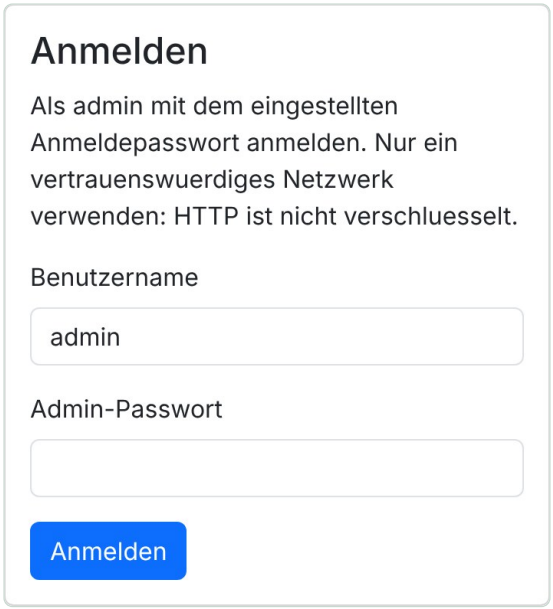


Abbildung 3: Anmeldeseite

> **Achtung**
> `ebctester` ist auf jedem Gerät gleich und steht in diesem Handbuch. Ändern Sie es beim ersten Gebrauch. Auch Geräte, die von Firmware 1.4.71 oder älter aktualisiert wurden, starten mit `ebctester`, solange noch kein Anmeldepasswort gespeichert ist.

Eine Anmeldung gilt je nach Einstellung 24 Stunden oder 7 Tage. Sie verlängert sich nicht durch Benutzung und endet, wenn das Gerät neu startet oder Sie auf **Abmelden** klicken. Laufende Tests und die Aufzeichnung sind vom An- und Abmelden nicht betroffen. Die Anmeldung lässt sich auch ganz abschalten (siehe Anmeldeeinstellungen).

2.3 Mit dem Heim-WLAN verbinden (optional)

EBCTester funktioniert vollständig ohne Heim-WLAN. Die Verbindung ist trotzdem sinnvoll: Sie erreichen das Gerät dann von jedem Gerät im Haus, die Uhrzeit wird aus dem Internet gestellt (nötig für korrekte Zeitstempel und die Update-Prüfung), und optionale Dienste wie MariaDB und MQTT werden möglich.

1. **Verwaltung → Einstellungen → WLAN einrichten** öffnen.
2. WLAN-Name (SSID) und Passwort eingeben und auf **Speichern** klicken. Auf Nachfrage mit dem Anmeldepasswort bestätigen.
3. Das Gerät verbindet sich; seine neue IP-Adresse steht unter **Geräteinformation → WLAN-IP**. Auch Ihr Router zeigt sie an.

Der geschützte Zugangspunkt bleibt als Rückfallweg aktiv. Unter `http://192.168.4.1` erreichen Sie das Gerät also immer, auch wenn das Heim-WLAN außer Reichweite ist. Ein Wechsel des WLAN-Kanals kann die Verbindung zum Zugangspunkt kurz unterbrechen.

2.4 Testermodell auswählen

Vor der ersten Verbindung müssen Sie EBCTester mitteilen, welchen Tester Sie verwenden – eine Voreinstellung gibt es nicht. Öffnen Sie **Verwaltung → Einstellungen → EBC tester**, wählen Sie das Modell und klicken Sie auf **Save tester model** (diese Karte ist in Firmware 1.4.72 noch nicht übersetzt).

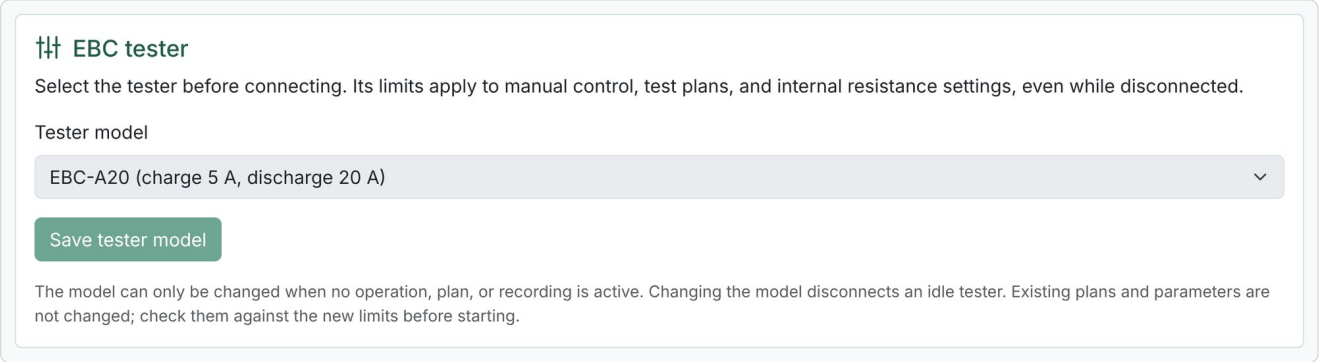


Abbildung 4: Testermodell auswählen

Das gewählte Modell und seine Grenzen gelten sofort, auch ohne angeschlossenen Tester. So können Sie Pläne und Parameter vorab vorbereiten. Das Modell lässt sich nur ändern, solange kein Vorgang, Plan und keine Aufzeichnung aktiv ist. Meldet der angeschlossene Tester ein anderes unterstütztes Modell, zeigt EBCTester einen Fehler, trennt die Verbindung und verbindet sich erst wieder automatisch, wenn Sie die Auswahl korrigieren oder erneut auf **Verbinden** klicken.

2.5 LEDs auf der Platine

Die EBCTester-Platine hat drei LEDs. LED1 und LED2 sind RGB-LEDs; an J5 lassen sich weitere LEDs anschließen, z. B. für den Gehäusedeckel.

LED	Anzeige	Bedeutung
D6 – Versorgung	(gelb-)grün, dauerhaft	Die 3,3-V-Versorgung ist in Ordnung. Bleibt D6 dunkel, 19-V- oder USB-Versorgung prüfen.
LED1 – Status	aus	Tester nicht verbunden
	grün	Tester verbunden, kein Vorgang aktiv
	blau	Manueller Betrieb läuft (Reiter Manuell)
	rot	Automatikbetrieb läuft (Testplan)
LED2 – RX	schwach weiß	Tester nicht verbunden oder länger als 5 s keine Meldung (in der Weboberfläche grau)
	dunkelgrün, dauerhaft	Tester verbunden, Meldungen kommen an
	hellgrün, kurz aufblitzend	Ein Aufblitzen je gültiger Meldung vom Tester
	orange	Kommunikationsfehler, z. B. ungültige Meldungen oder ein Tester, der nicht zum gewählten Modell passt

LED2 verwendet dieselben Farben wie die RX-Anzeige in der Kopfzeile der Weboberfläche. Dort bedeutet Gelb, dass der *Browser* das Gerät nicht erreicht; das kann die LED auf der Platine nicht anzeigen. Sie zeigt stattdessen Orange bei Fehlern in der Kommunikation mit dem Tester.

> **Hinweis**
> Die hier beschriebenen Funktionen von LED1 und LED2 sind für eine kommende Firmware-Version vorgesehen. Firmware 1.4.72 unterstützt sie noch nicht; sie zeigt auf LED1 nur den Zustand von Firmware-Updates an (siehe Abschnitt 9.5). D6 arbeitet unabhängig von der Firmware.

2.6 Die Weboberfläche im Überblick



Abbildung 5: Kopfzeile

Die Kopfzeile ist immer sichtbar. Von links nach rechts enthält sie:

Logo	Führt zur Startseite.
Modell	Das gewählte Testermodell (z. B. *EBC-A20*). Es wird zur Warnung, wenn der angeschlossene Tester nicht passt oder unbekannt ist.
Batterietester / Verwaltung	Schaltet zwischen den beiden Bereichen der Oberfläche um. Jeder Bereich merkt sich den zuletzt geöffneten Reiter.
RX-Anzeige	**Dunkelgrün**, solange der Tester verbunden ist, mit einem hellgrünen Aufblitzen je gültiger Meldung vom Tester. **Grau** = Tester nicht verbunden, **gelb** = der Browser erreicht das Gerät nicht. LED2 auf der Platine zeigt dasselbe an (siehe Abschnitt 2.5).
Abmelden	Beendet die Browser-Sitzung. Ausgeblendet, wenn die Browser-Anmeldung abgeschaltet ist.

Sprache	English, Deutsch und alle von der SD-Karte geladenen Sprachen. Die Auswahl wird im Browser gespeichert; ein Wechsel lädt die Seite neu (nicht gespeicherte Eingaben gehen verloren, ein laufender Test ist nicht betroffen).

Bereich	Reiter
Batterietester	Messwerte · Batterie · Manuell · Automatik · Innenwiderstand
Verwaltung	Geräteinformation · Testlaufverwaltung · SD-Karte · Updates · Einstellungen · Debug (nur wenn eingeschaltet)

Zahlenfelder werden geprüft, sobald Sie sie verlassen. Eine rote Meldung unter dem Feld nennt den erlaubten Bereich; Start- und Speicherknöpfe bleiben gesperrt, bis alle Felder gültig sind. Ist ein Startknopf aus einem anderen Grund gesperrt (z. B. Tester nicht verbunden), steht der Grund in einem gelben Hinweis darunter.

> **Hinweis**
> Die Bildschirmfotos in diesem Handbuch zeigen Beispieldaten: einen EBC-A20, der eine LiFePO4-Zelle mit 50 Ah testet. Es sind Beispieleinstellungen, keine Empfehlungen für Ihren Akku. Einige Texte erscheinen in Firmware 1.4.72 noch ohne Umlaute (z. B. „pruefen“) oder auf Englisch; im Handbuch sind die Bezeichnungen mit Umlauten geschrieben.

3 Bereich Batterietester

3.1 Messwerte

Der Reiter **Messwerte** ist die Übersicht über aufgezeichnete Testläufe. Alle Daten stammen aus der Datenbank auf der SD-Karte; die Seite funktioniert deshalb auch am Zugangspunkt ohne Internet.

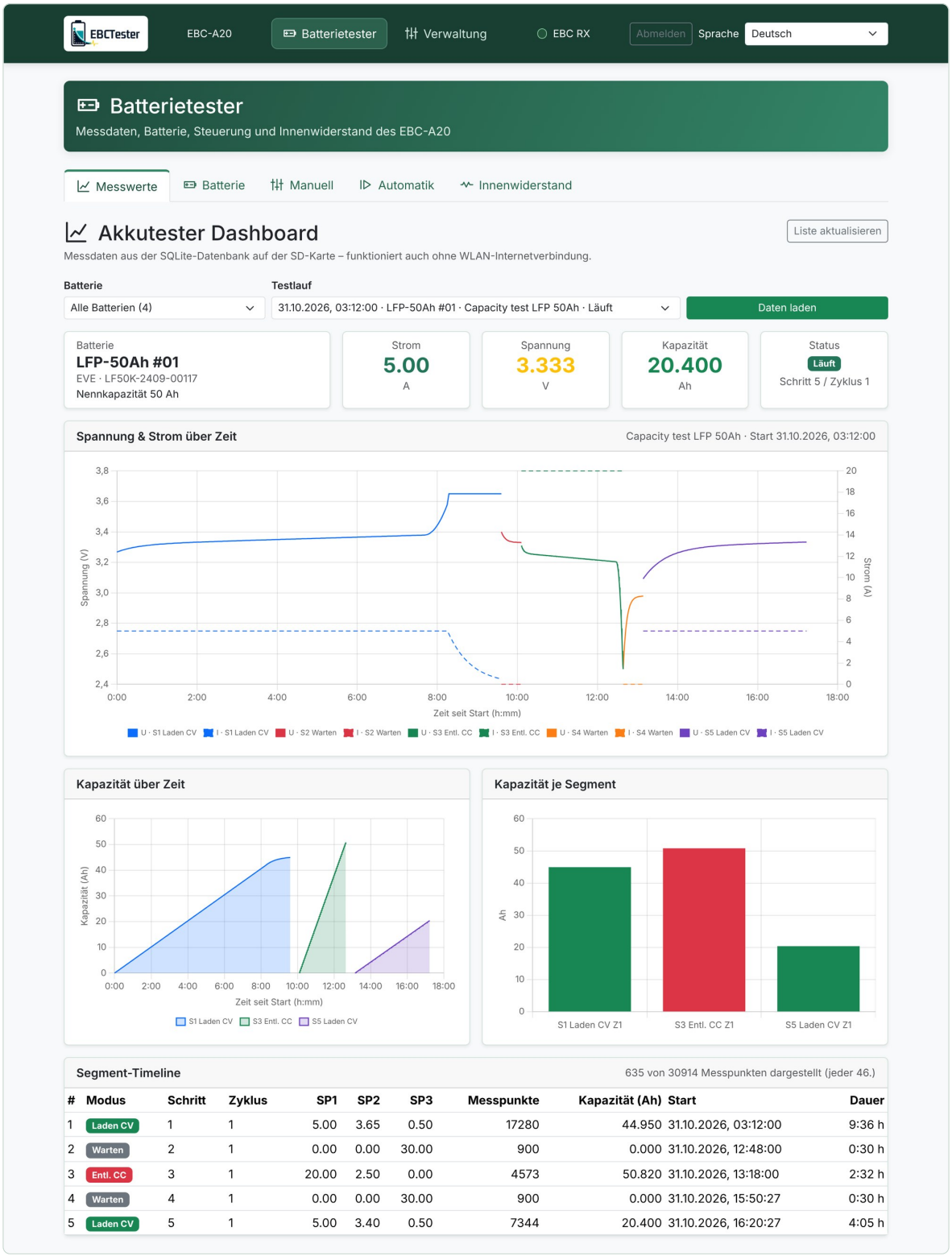


Abbildung 6: Messwerte während eines laufenden Kapazitätstests

Testlauf auswählen

1. Die Liste bei Bedarf mit **Batterie** eingrenzen.
2. Unter **Testlauf** einen Lauf wählen. Die Läufe stehen mit dem neuesten zuerst, mit Datum, Batterie, Plan und Status (oder mit dem in der Testlaufverwaltung vergebenen Namen).
3. Auf **Daten laden** klicken. **Liste aktualisieren** liest die Liste der Läufe neu ein, z. B. nachdem ein neuer Test gestartet wurde.

Beim Öffnen der Seite wird der neueste Lauf automatisch geladen. Ein laufender Test wird alle 15 Sekunden aktualisiert.

Was angezeigt wird

- **Übersichtskarten:** Batterie (Bezeichnung, Hersteller, Seriennummer, Nennkapazität), der letzte Strom-, Spannungs- und Kapazitätswert sowie der Status des Laufs mit aktuellem Schritt und Zyklus.
- **Spannung & Strom über Zeit:** eine Farbe je Segment; Spannung als durchgezogene Linie (linke Achse), Strom gestrichelt (rechte Achse). Die Zeitachse zeigt Stunden:Minuten seit Start des Laufs.
- **Kapazität über Zeit** für jedes Lade- oder Entladesegment und **Kapazität je Segment** als Balkendiagramm (grün = Laden, rot = Entladen).
- **Segment-Timeline:** eine Zeile je Segment mit Modus, Planschritt, Zyklus, den drei Sollwerten (SP1–SP3), Anzahl der Messpunkte, Kapazität, Startzeit und Dauer.

Ein *Segment* ist ein zusammenhängender Abschnitt eines Laufs mit einer Betriebsart – zum Beispiel „Laden CC/CV“ oder „Warten“. Damit die Diagramme schnell bleiben, werden höchstens 1500 Punkte gezeichnet; die Zeile über der Timeline nennt, wie viele der gespeicherten Punkte dargestellt sind („jeder n-te“). Segmentgrenzen sind immer enthalten. Fahren Sie mit der Maus über ein Diagramm (oder tippen Sie darauf), um die genauen Werte zu sehen.

Status	Bedeutung
Läuft	Der Test wird noch aufgezeichnet.
Abgeschlossen	Normal beendet (Plan fertig oder Abschaltbedingung erreicht).
Gestoppt	Vom Benutzer gestoppt.
Unterbrochen	Durch Stromausfall, Neustart oder Verbindungsverlust beendet.
Fehler	Wegen eines Fehlers beendet (Einzelheiten im Debug-Protokoll).

3.2 Batterie

Hier erfassen Sie den Akku, der als Nächstes getestet wird. Seine Daten werden mit jedem Testlauf gespeichert; so wissen Sie immer, zu welcher Zelle oder welchem Pack eine Messung gehört.

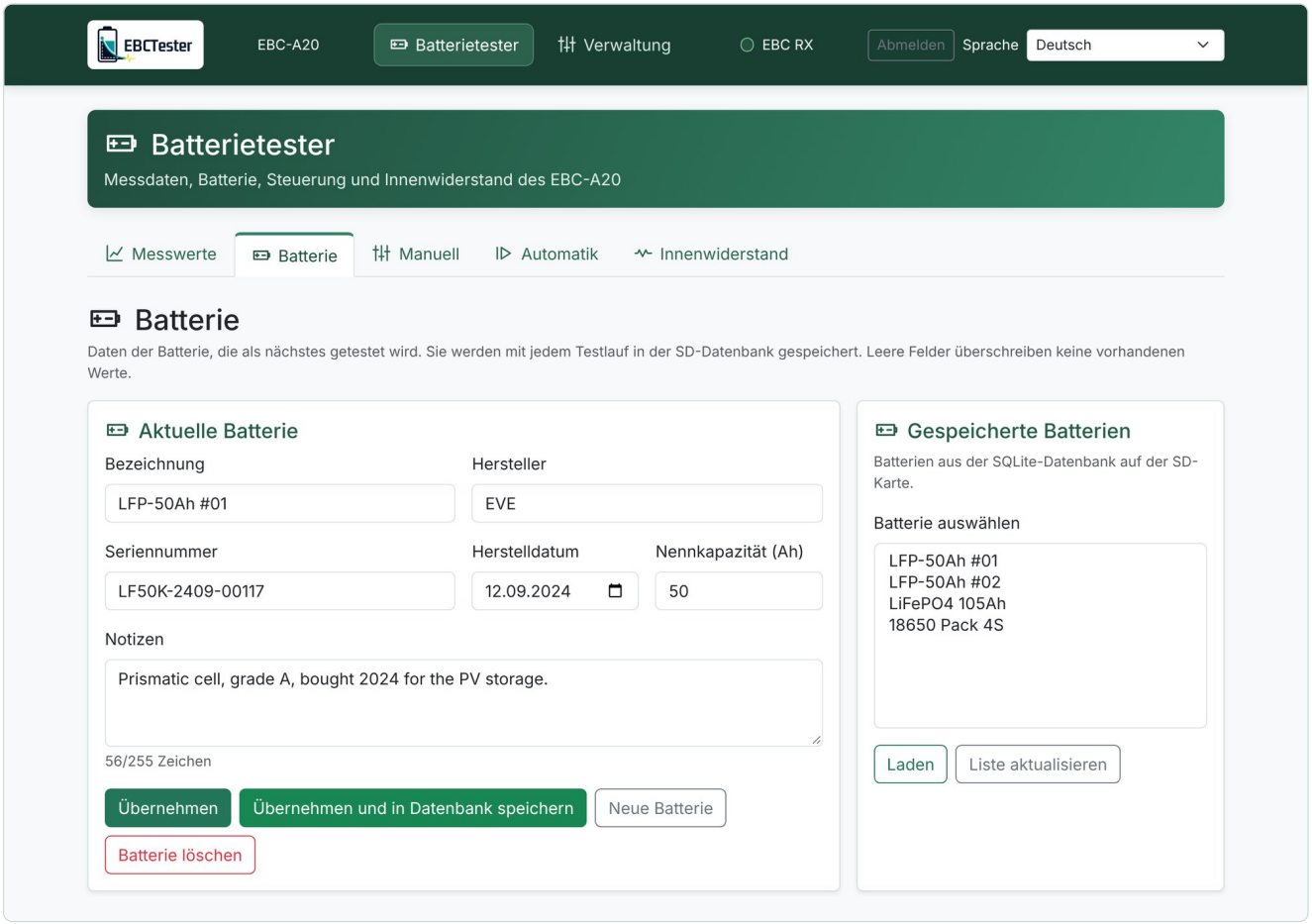


Abbildung 7: Reiter Batterie

Feld	Beschreibung
Bezeichnung	Eindeutiger Name des Akkus, z. B. *LFP-50Ah #01*. Pflicht für das Speichern in der Datenbank.
Hersteller, Seriennummer	Freitext.
Herstelldatum	Datumsauswahl.
Nennkapazität (Ah)	Angegebene Kapazität; wird zum Vergleich in den Messwerten angezeigt.
Notizen	Bis zu 255 Zeichen.

Knopf	Funktion
Übernehmen	Macht den eingegebenen Akku zur aktuellen Batterie (auf dem Gerät gespeichert). Der nächste Testlauf wird damit aufgezeichnet.
Übernehmen und in Datenbank speichern	Speichert die Batterie zusätzlich in der SD-Datenbank, damit sie später wieder auswählbar ist.
Neue Batterie	Leert das Formular für einen neuen Eintrag.
Batterie löschen	Löscht die im Formular angezeigte gespeicherte Batterie nach Bestätigung und Eingabe des Anmeldepassworts. Nicht möglich, solange aufgezeichnete Testläufe sie verwenden – diese zuerst in der Testlaufverwaltung löschen – oder solange eine Aufzeichnung, ein Plan oder eine DCIR-Messung aktiv ist. MariaDB-Sicherungen bleiben unverändert.
Laden / Liste aktualisieren	Unter **Gespeicherte Batterien** eine Batterie wählen und auf **Laden** klicken (oder den Eintrag doppelklicken), um sie ins Formular zu übernehmen. **Liste aktualisieren** liest die Datenbank neu ein.

Leere Felder überschreiben keine bereits gespeicherten Werte.

3.3 Manuelle Bedienung

Im Reiter **Manuell** steuern Sie den Tester direkt – vergleichbar mit den Tasten am Tester selbst, aber mit Aufzeichnung.

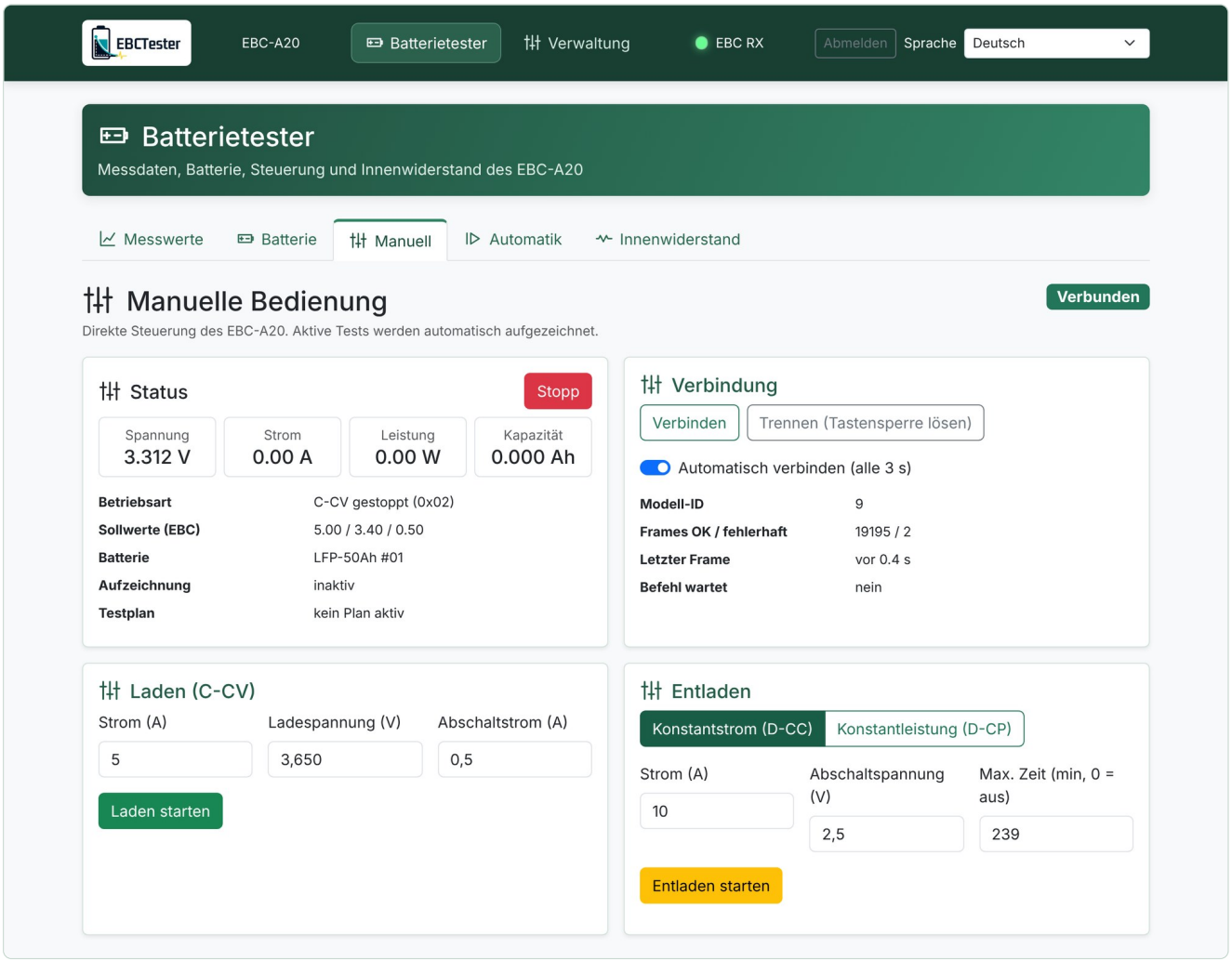


Abbildung 8: Manuelle Bedienung bei verbundenem Tester ohne aktiven Vorgang

Status

Zeigt die aktuellen Werte des Testers (Spannung, Strom, Leistung, Kapazität), die Betriebsart mit ihrem Code, die im Tester aktiven Sollwerte, die aktuelle Batterie, den Stand der Aufzeichnung (Anzahl gespeicherter Messwerte) und ob ein Testplan aktiv ist. Das Kennzeichen oben rechts zeigt *Getrennt*, *Verbunden* oder die aktive Betriebsart. **Stopp** beendet den laufenden Vorgang sofort und ist bei bestehender Verbindung immer verfügbar.

Verbindung

- **Verbinden** startet die Kommunikation mit dem Tester. Solange die Verbindung besteht, ist das Tastenfeld des Testers gesperrt.
- **Trennen (Tastensperre lösen)** beendet die Verbindung und gibt das Tastenfeld wieder frei.
- **Automatisch verbinden (alle 3 s)**: EBCTester verbindet sich selbstständig, z. B. nach dem Einschalten des Testers.

- Diagnosewerte: vom Tester gemeldete Modell-ID, Anzahl gültiger/fehlerhafter Meldungen, Alter der letzten Meldung und ob ein Befehl auf Bestätigung wartet.

Laden (C-CV)

Lädt mit konstantem Strom, bis die Ladespannung erreicht ist, danach mit konstanter Spannung, bis der Strom unter den Abschaltstrom fällt.

Feld	Bedeutung	Bereich EBC-A20
Strom (A)	Ladestrom in der CC-Phase	0,01 – 5 A
Ladespannung (V)	Ladeschlussspannung (CV-Pegel)	bis 18 V
Abschaltstrom (A)	Das Laden endet, wenn der Strom unter diesen Wert fällt	≤ Ladestrom

Entladen

Konstantstrom (D-CC) oder **Konstantleistung (D-CP)** wählen.

Feld	Bedeutung	Bereich EBC-A20
Strom (A) / Leistung (W)	Entladestrom (D-CC) bzw. -leistung (D-CP)	0,01 – 20 A / 1 – 85 W
Abschaltspannung (V)	Das Entladen endet bei dieser Akkuspannung	bis 30 V
Max. Zeit (min, 0 = aus)	Zusätzliche Zeitgrenze	0 – 239 min

Im Modus D-CP muss die Leistung auch mit dem Höchststrom bei Abschaltspannung erreichbar sein (Leistung ≤ max. Strom × Abschaltspannung). Für A40L/A50L gelten die Bereiche des jeweiligen Modells.

Nach **Laden starten** oder **Entladen starten** wartet EBCTester bis zu 10 Sekunden, bis der Tester die Betriebsart bestätigt; andernfalls meldet es einen Fehler und sendet Stopp. Jeder manuelle Vorgang wird automatisch als Testlauf aufgezeichnet.

> **Hinweis**
> Die Startknöpfe sind gesperrt, solange ein Befehl auf Bestätigung wartet, ein anderer Vorgang oder ein Testplan läuft (auch während seiner Warteschritte) oder kein Tester verbunden ist. Der Grund steht unter dem Knopf.

3.4 Automatische Bedienung (Testpläne)

Ein Testplan ist eine Folge von bis zu **10 Schritten**, die EBCTester selbstständig abarbeitet. Der Plan läuft auf dem ESP32 und läuft ohne WLAN und ohne geöffneten Browser weiter, solange das Gerät Strom hat.

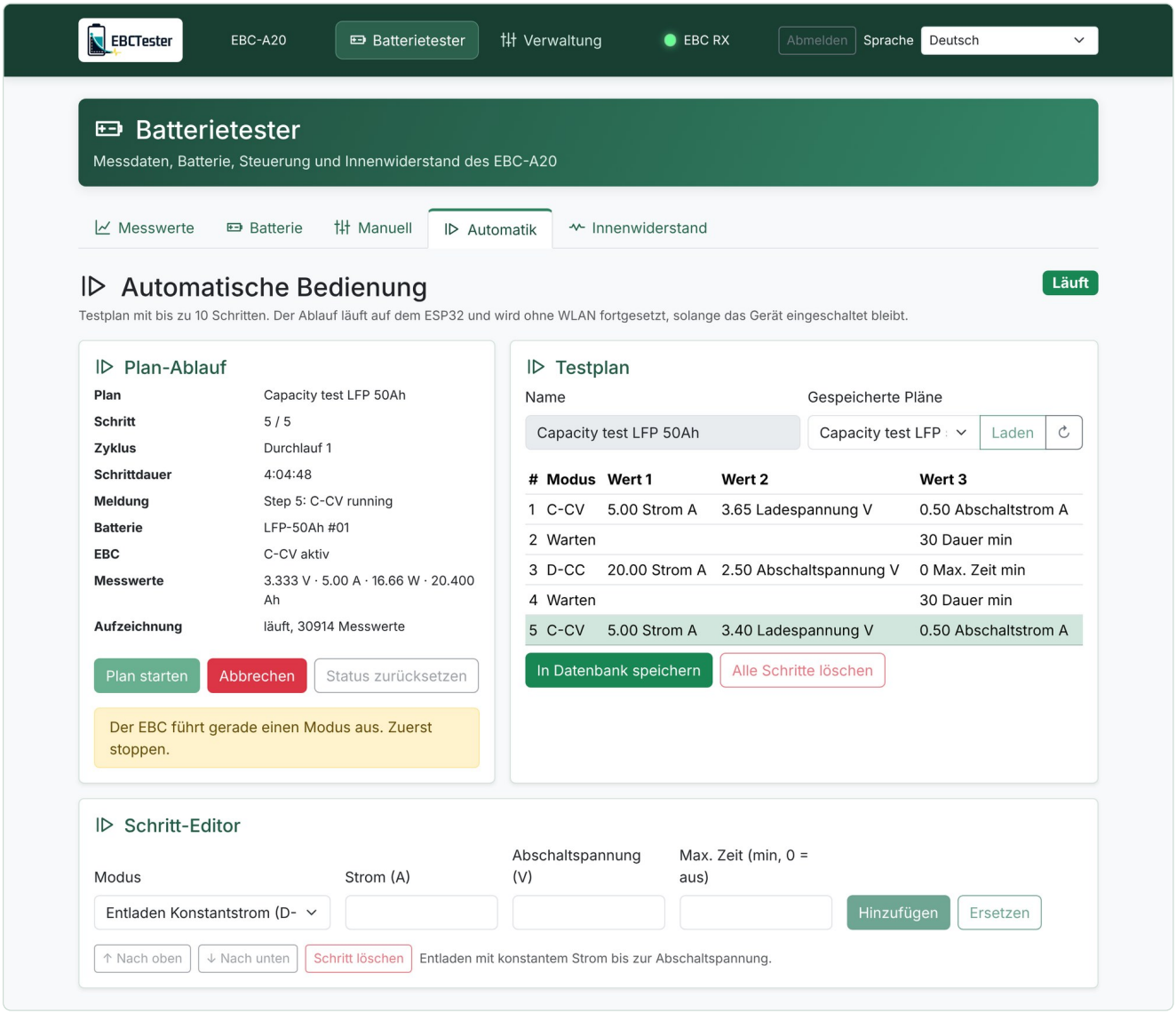


Abbildung 9: Automatische Bedienung: laufender Plan (links) und Plan-Editor (rechts)

Schrittarten

Modus	Wert 1	Wert 2	Wert 3	Funktion
D-CC	Strom (A)	Abschaltspannung (V)	Max. Zeit (min, 0 = aus)	Entladen mit konstantem Strom bis zur Abschaltspannung.
D-CP	Leistung (W)	Abschaltspannung (V)	Max. Zeit (min, 0 = aus)	Entladen mit konstanter Leistung bis zur Abschaltspannung.
C-CV	Strom (A)	Ladespannung (V)	Abschaltstrom (A)	Laden mit CC/CV bis zum Abschaltstrom.
Warten	–	–	Dauer (1 – 239 min)	Pause, z. B. damit sich die Zelle erholt.
Zyklus	–	–	Durchläufe gesamt (1 – 239)	Springt zurück zu Schritt 1, bis die Anzahl der Durchläufe erreicht ist. Nicht als erster Schritt erlaubt und nur einmal je Plan.

Plan erstellen und bearbeiten

1. Einen **Namen** für den Plan eingeben.
2. Im **Schritt-Editor** den Modus wählen, die Werte eintragen und auf **Hinzufügen** klicken. Der Schritt wird angehängt – oder hinter der ausgewählten Zeile eingefügt.

3. Eine Zeile der Tabelle anklicken, um sie auszuwählen. Dann können Sie sie durch die Werte im Editor ersetzen (**Ersetzen**), verschieben (**↑ Nach oben**, **↓ Nach unten**) oder löschen (**Schritt löschen**). Ein erneuter Klick hebt die Auswahl auf.
4. **In Datenbank speichern** legt den Plan unter seinem Namen auf der SD-Karte ab. **Alle Schritte löschen** leert die Tabelle.

Jede Änderung wird sofort auf dem Gerät gespeichert und gegen die Grenzen des gewählten Testers geprüft. Gespeicherte Pläne laden Sie über die Liste **Gespeicherte Pläne** und **Laden**; ↻ aktualisiert die Liste.

Beispiel: Kapazitätstest einer LiFePO4-Zelle

#	Modus	Werte	Zweck
1	C-CV	5,00 A · 3,65 V · 0,50 A	Vollladen
2	Warten	30 min	Zelle ruhen lassen
3	D-CC	20,00 A · 2,50 V · 0 (ohne Zeitgrenze)	Kapazität messen
4	Warten	30 min	Zelle erholen lassen
5	C-CV	5,00 A · 3,40 V · 0,50 A	Auf Lagerspannung laden

Nur Beispielwerte – verwenden Sie die Werte aus dem Datenblatt Ihrer Zelle.

Plan ausführen

Tester verbinden, die aktuelle Batterie prüfen und auf **Plan starten** klicken. Die Karte **Plan-Ablauf** zeigt Planname, aktuellen Schritt, Zyklus, Zeit im Schritt, Statusmeldung, Batterie, Betriebsart des Testers, aktuelle Messwerte und die Zahl der aufgezeichneten Messwerte. Der aktive Schritt ist in der Tabelle grün markiert.

- **Abbrechen** stoppt nach Bestätigung den Plan und den Tester.
- **Status zurücksetzen** setzt einen abgeschlossenen, abgebrochenen oder unterbrochenen Plan auf „Bereit“, damit er erneut gestartet werden kann.

> **Achtung**
> Nach einem Stromausfall oder Neustart wird ein laufender Plan **nicht** automatisch fortgesetzt. Er wird als *Unterbrochen* markiert. Prüfen Sie den Akku, klicken Sie dann auf **Status zurücksetzen** und starten Sie bei Bedarf neu.

3.5 Innenwiderstand (DCIR)

Dieser Reiter misst den Gleichstrom-Innenwiderstand eines Akkus mit einem Lastpuls. Der Akku ruht ohne Last, dann wird ein festgelegter Entladestrom eingeschaltet. Aus dem Spannungseinbruch wird der Widerstand dreimal berechnet – standardmäßig 1, 5 und 10 Sekunden nach dem Einschalten:

$$R = (U_1 - U_2) / I_2$$

U_1 ist die gemittelte Ruhespannung am Ende der Ruhephase, U_2 und I_2 sind Spannung und *gemessener* Strom zum Messzeitpunkt. Kurze Zeiten bilden vor allem den ohmschen Widerstand ab (Kontakte, Elektrolyt), längere Zeiten enthalten zusätzlich Polarisationseffekte.

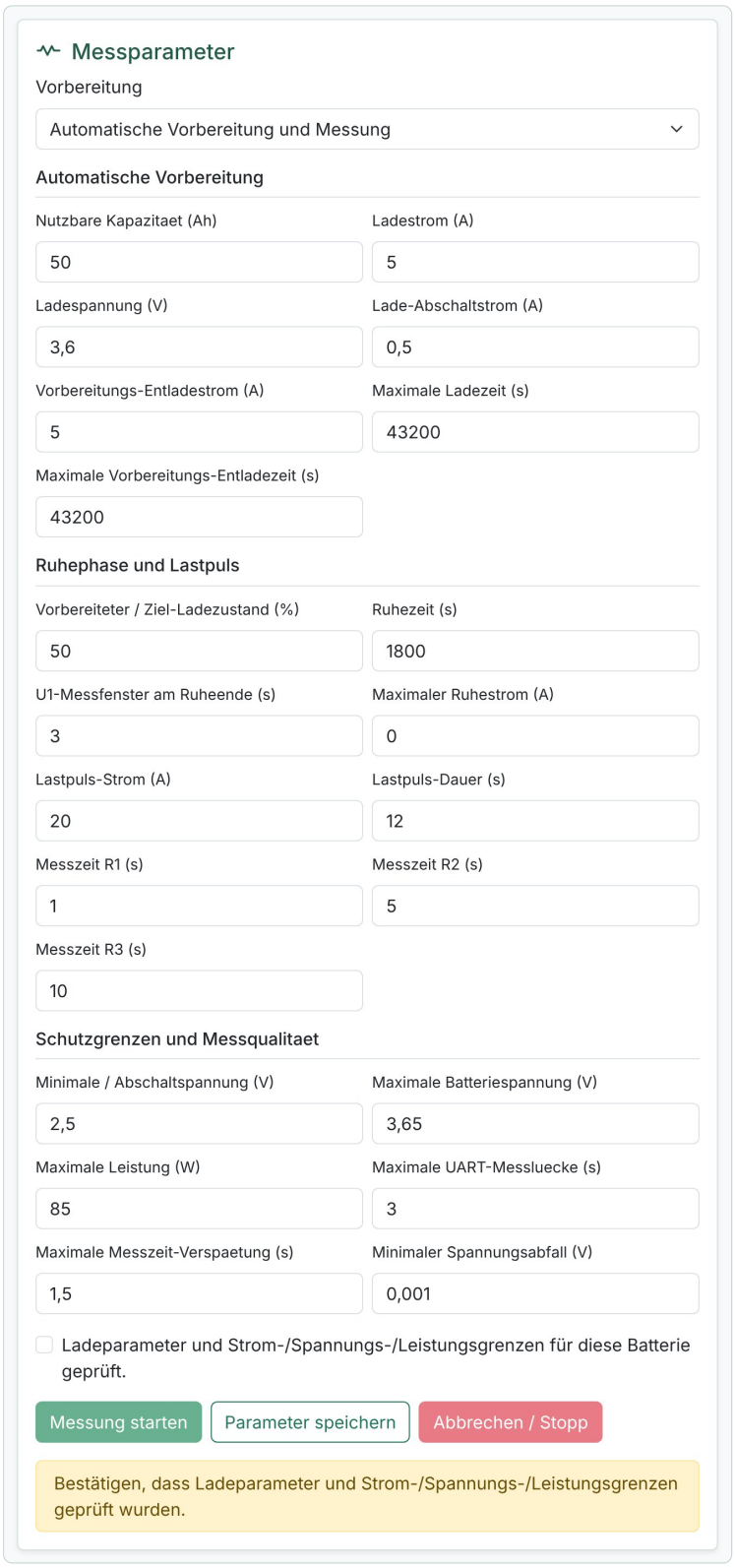


Abbildung 10: Messparameter

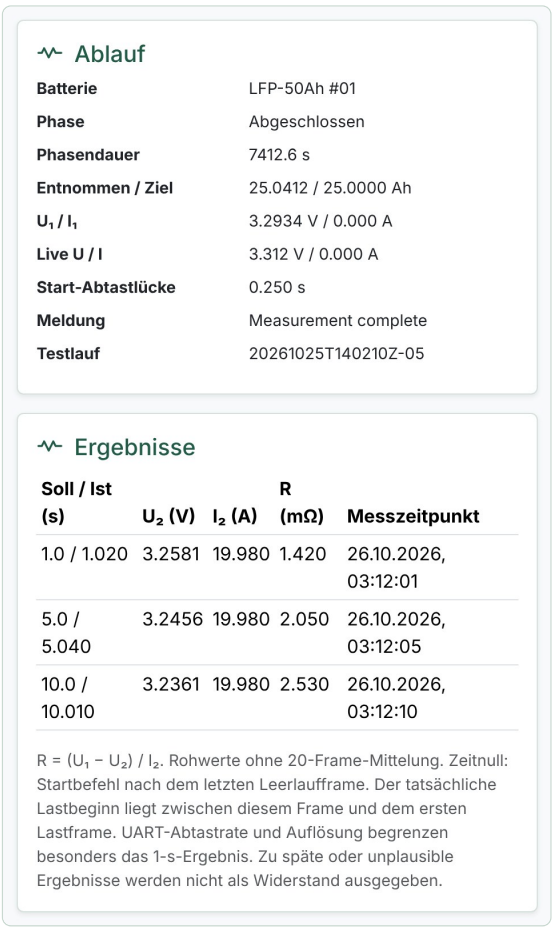


Abbildung 11: Ablauf und Ergebnisse einer abgeschlossenen Messung

Vorbereitung

- **Ladezustand bereits vorbereitet:** Der Akku hat bereits den gewünschten Ladezustand. Die Messung beginnt mit der Ruhephase.
- **Automatische Vorbereitung und Messung:** Der Akku wird zuerst voll geladen (C-CV), danach wird die berechnete Ladungsmenge entnommen: *nutzbare Kapazität × (100 − Ziel-SOC) / 100*. Die entnommene Ladung wird aus dem gemessenen Strom aufsummiert. Der Ziel-Ladezustand ist daher ein Schätzwert auf Basis der eingegebenen Kapazität, kein Messwert. Bei 100 % entfällt die Entladung.

Parameter

Gruppe / Parameter	Bedeutung	Standard
Automatische Vorbereitung (nur bei automatischer Vorbereitung)		
Nutzbare Kapazität (Ah)	Grundlage für die zu entnehmende Ladung	20
Ladestrom / Ladespannung / Lade-Abschaltstrom	Vollladung vor der Messung	0,5 A / 3,45 V / 0,1 A
Vorbereitungs-Entladestrom (A)	Strom, mit dem der Ziel-Ladezustand angefahren wird	0,5
Maximale Lade- / Vorbereitungs-Entladezeit (s)	Zeitgrenzen der beiden Phasen	86400
Ruhephase und Lastpuls		
Vorbereiteter / Ziel-Ladezustand (%)	Ziel-Ladezustand (automatisch) bzw. Angabe (vorbereitet)	50
Ruhezeit (s)	Ruhe ohne Last vor dem Puls	60
U1-Messfenster am Ruheende (s)	Mittelungsfenster für U_1	3
Maximaler Ruhestrom (A)	Strom, der noch als „ohne Last“ gilt	0,02
Lastpuls-Strom (A) / Lastpuls-Dauer (s)	Lastpuls	0,5 A / 12 s
Messzeit R1 / R2 / R3 (s)	Zeitpunkte der drei Ergebnisse nach dem Einschalten	1 / 5 / 10
Schutzgrenzen und Messqualität		
Minimale / Abschaltspannung (V)	Unterhalb dieser Spannung wird abgebrochen	2,5
Maximale Batteriespannung (V)	Oberhalb dieser Spannung wird abgebrochen	3,6
Maximale Leistung (W)	Obergrenze für Strom × Maximalspannung	50
Maximale UART-Messlücke (s)	Längster erlaubter Abstand zwischen Testermeldungen	3
Maximale Messzeit-Verspätung (s)	Zeittoleranz der Messpunkte	1,5
Minimaler Spannungsabfall (V)	Kleinere Einbrüche gelten als nicht plausibel	0,001

Prüfregeln: Die Messzeiten müssen aufsteigen; der Puls muss mindestens so lange dauern wie die letzte Messzeit plus Zeittoleranz; die Ruhezeit muss mindestens so lang sein wie das U1-Messfenster; Abschalt- und Ladespannung müssen unter der Maximalspannung liegen; Strom × Maximalspannung darf die maximale Leistung nicht überschreiten. Außerdem müssen alle Werte innerhalb der Grenzen des gewählten Testers liegen.

Messung durchführen

1. Akku mit **Vierleitermessung** anschließen (getrennte Messleitungen) – sonst geht der Kabelwiderstand ins Ergebnis ein.

2. Alle Parameter prüfen und auf **Parameter speichern** klicken.
3. Das Bestätigungsfeld ankreuzen („Ladeparameter und Strom-/Spannungs-/Leistungsgrenzen für diese Batterie geprüft“). Jede Parameteränderung entfernt das Häkchen wieder.
4. Auf **Messung starten** klicken. Die SD-Datenbank muss bereit sein.
5. Den Fortschritt in der Karte **Ablauf** verfolgen; **Abbrechen / Stopp** beendet die Messung jederzeit.

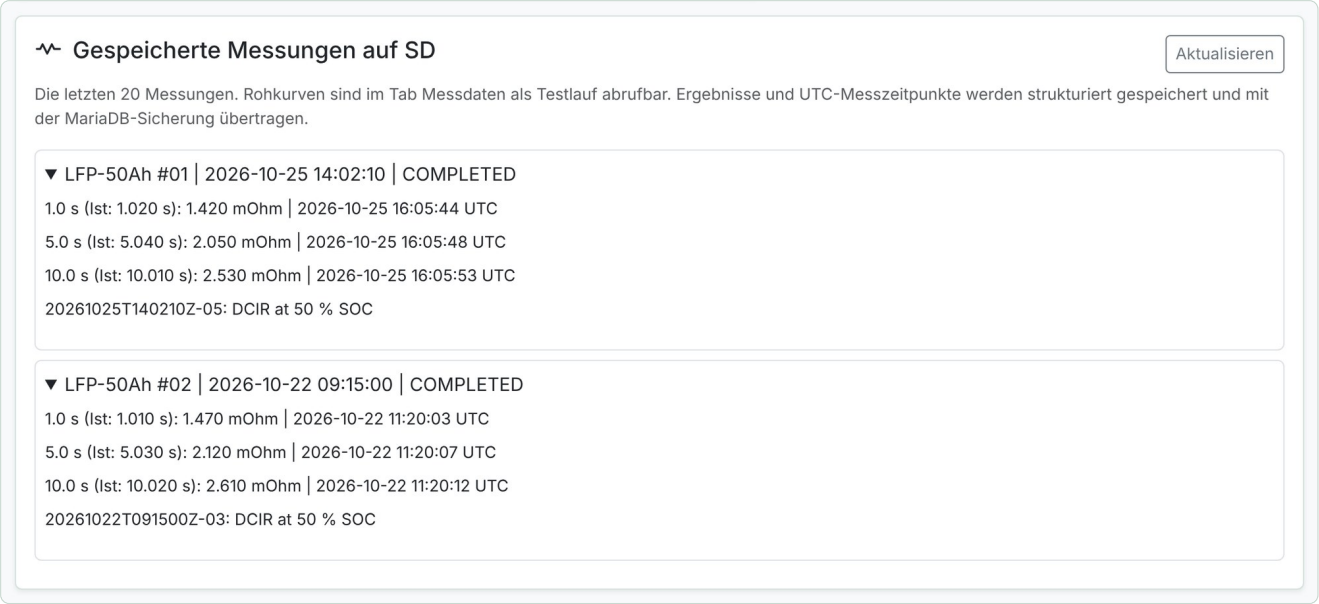


Abbildung 12: Gespeicherte Messungen auf SD

Die Tabelle **Ergebnisse** nennt für jeden Messzeitpunkt die angeforderte und die tatsächliche Zeit, U_2, I_2, den Widerstand in mΩ und den Zeitstempel. Verspätete oder fehlende Messpunkte, zu kleine oder negative Spannungseinbrüche oder fehlender Strom ergeben ein „ungültiges“ Ergebnis statt einer falschen Zahl. Die *Start-Abtastlücke* zeigt die Unsicherheit des Einschaltzeitpunkts, die durch den Abstand der Testermeldungen entsteht – sie begrenzt vor allem die Genauigkeit des 1-Sekunden-Werts.

Jede DCIR-Messung wird als normaler Testlauf (Rohkurve unter **Messwerte**) und zusätzlich mit strukturierten Ergebnissen gespeichert. **Gespeicherte Messungen auf SD** listet die letzten 20 DCIR-Messungen. Die Temperatur wird nicht gemessen und als unbekannt ausgewiesen.

> **Hinweis**
> Während einer DCIR-Messung sind alle anderen Starts, Verbinden/Trennen und Batteriewechsel gesperrt. Stopp und Abbrechen bleiben immer verfügbar. Eine DCIR-Messung wird nach einem Neustart nie fortgesetzt.

4 Bereich Verwaltung

4.1 Geräteinformation

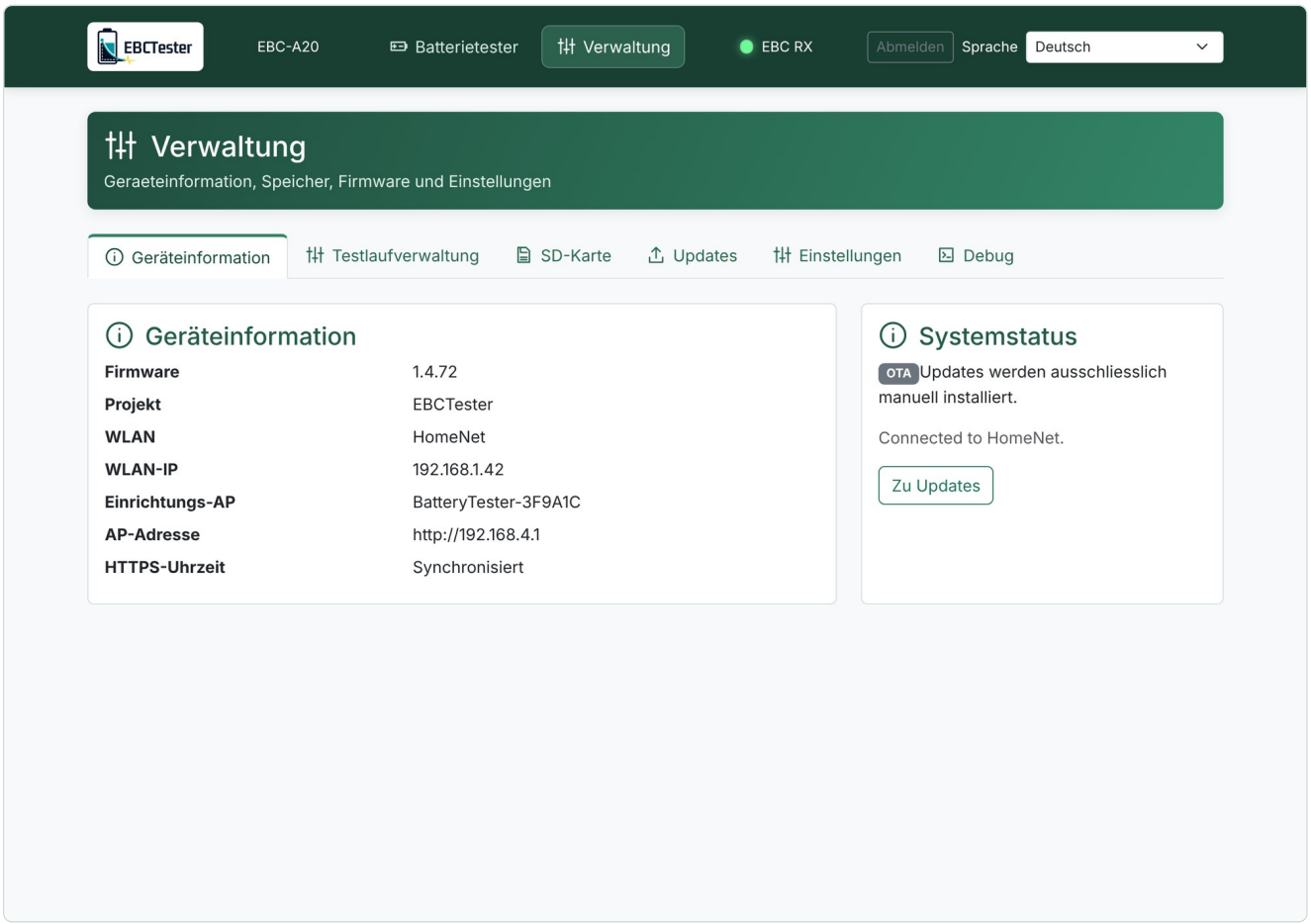


Abbildung 13: Geräteinformation

Zeigt die installierte Firmware-Version, den Projektnamen, das verbundene WLAN und die IP-Adresse, den Namen des Einrichtungs-Zugangspunkts (`BatteryTester-XXXXXX`), seine Adresse `http://192.168.4.1` und ob die Uhr synchronisiert ist („HTTPS-Uhr“). **Systemstatus** zeigt die Netzwerkmeldung und führt zur Update-Seite.

4.2 Testlaufverwaltung

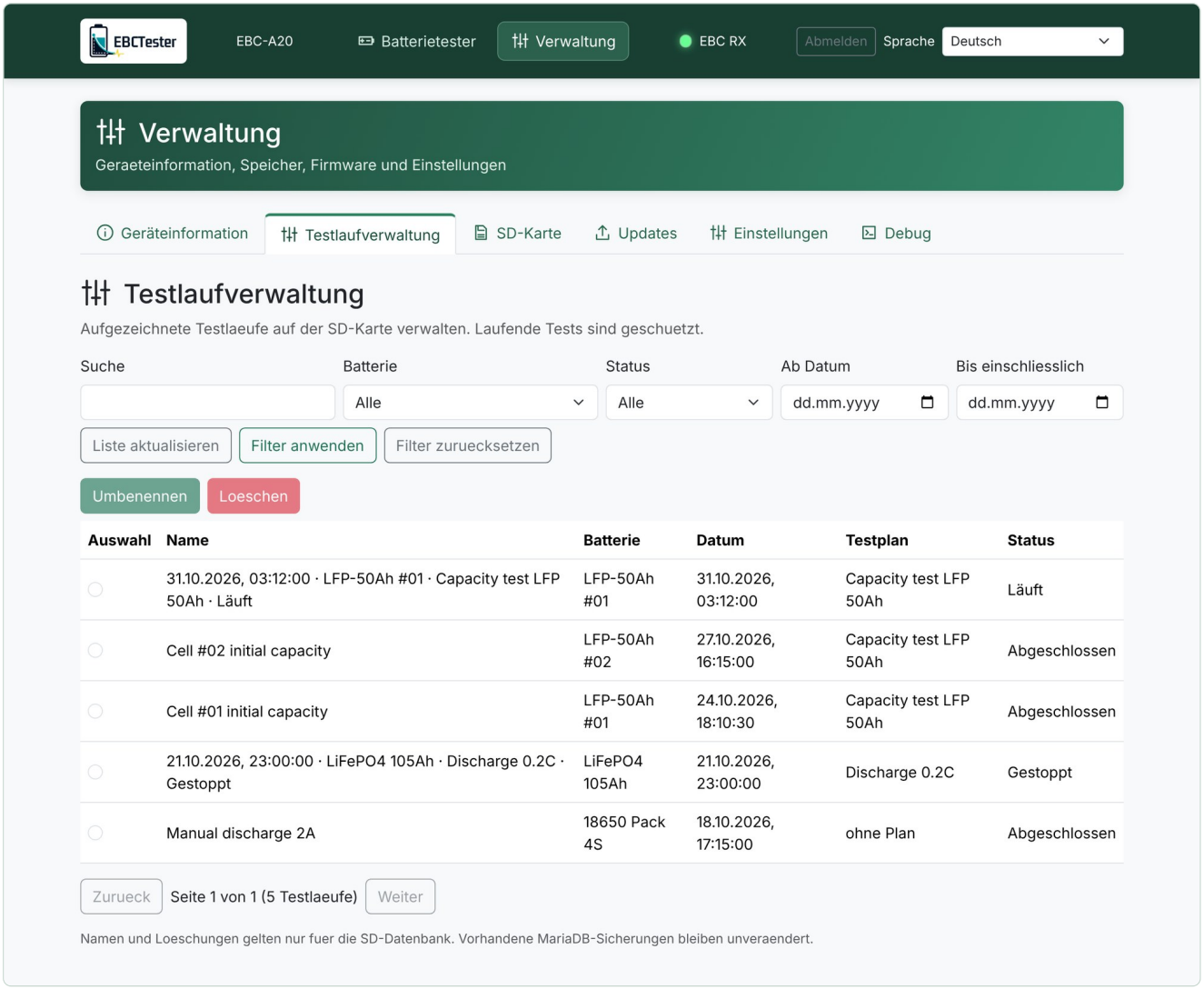


Abbildung 14: Testlaufverwaltung

Diese Tabelle listet *alle* Läufe der SD-Datenbank, 50 je Seite, die neuesten zuerst.

- **Filter:** Freitext-**Suche** (Name, Batterie, Plan, Datum, Status oder Lauf-ID), **Batterie**, **Status** und ein Datumsbereich (**Ab Datum** / **Bis einschließlich**; das Enddatum schließt den ganzen Tag ein). Auf **Filter anwenden** klicken; **Filter zurücksetzen** löscht sie.
- **Umbenennen:** einen Lauf mit dem Auswahlknopf markieren und auf **Umbenennen** klicken. Bis zu 100 Zeichen; ein leerer Name stellt den automatischen Namen wieder her. Der Name wird auch in den Messwerten verwendet.
- **Löschen:** einen Lauf markieren und auf **Löschen** klicken. Nach Bestätigung wird der Lauf mit seinen Messwerten, Segmenten und DCIR-Ergebnissen endgültig entfernt. Batterien und Pläne bleiben erhalten.

Laufende Testläufe sind geschützt und lassen sich weder umbenennen noch löschen. Umbenennen und Löschen erfordern das Anmeldepasswort und betreffen nur die SD-Datenbank – vorhandene MariaDB-Sicherungen bleiben unverändert, und eigene Namen werden nicht nach MariaDB übertragen.

Achtung
Löschen lässt sich nicht rückgängig machen. Legen Sie vor dem Löschen wichtiger Läufe eine Sicherung an (MariaDB-Synchronisation oder Kopie der Datenbankdatei), und löschen Sie keine Läufe, während eine MariaDB-Synchronisation läuft.

4.3 SD-Karte

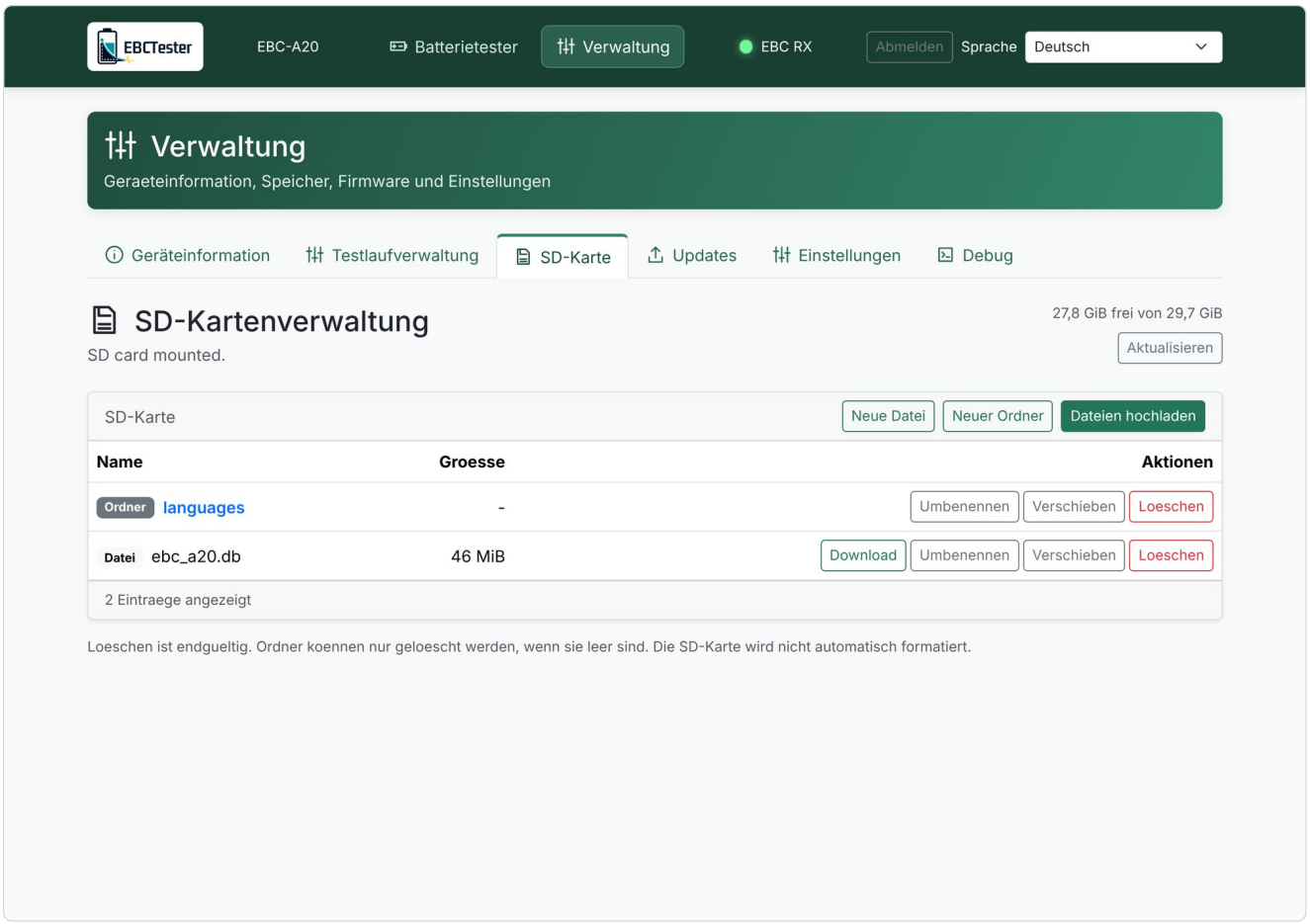


Abbildung 15: Verwaltung der SD-Karte

Ein einfacher Dateimanager für die microSD-Karte – die Karte muss dafür nicht aus dem Gerät genommen werden. Die Kopfzeile zeigt, ob eine Karte eingebunden ist und wie viel Platz frei ist.

- Einen Ordnernamen anklicken, um ihn zu öffnen; über den Pfad oben geht es zurück.
- **Neue Datei**, **Neuer Ordner** und **Dateien hochladen** (mehrere Dateien möglich, mit Fortschrittsbalken).
- Für jeden Eintrag: **Download** (nur Dateien), **Umbenennen**, **Verschieben** und **Löschen**.

Löschen ist endgültig; nur leere Ordner lassen sich löschen. Die Karte wird nie automatisch formatiert. Die Messdatenbank ist die Datei `ebc_a20.db` im Hauptverzeichnis; laden Sie sie regelmäßig als Sicherung herunter.

Achtung
Die Datenbankdatei nicht umbenennen, verschieben oder löschen, während das Gerät in Gebrauch ist. Laden Sie sie nur herunter, wenn gerade kein Test aufgezeichnet wird – nur dann ist die Kopie in sich stimmig.

4.4 Updates

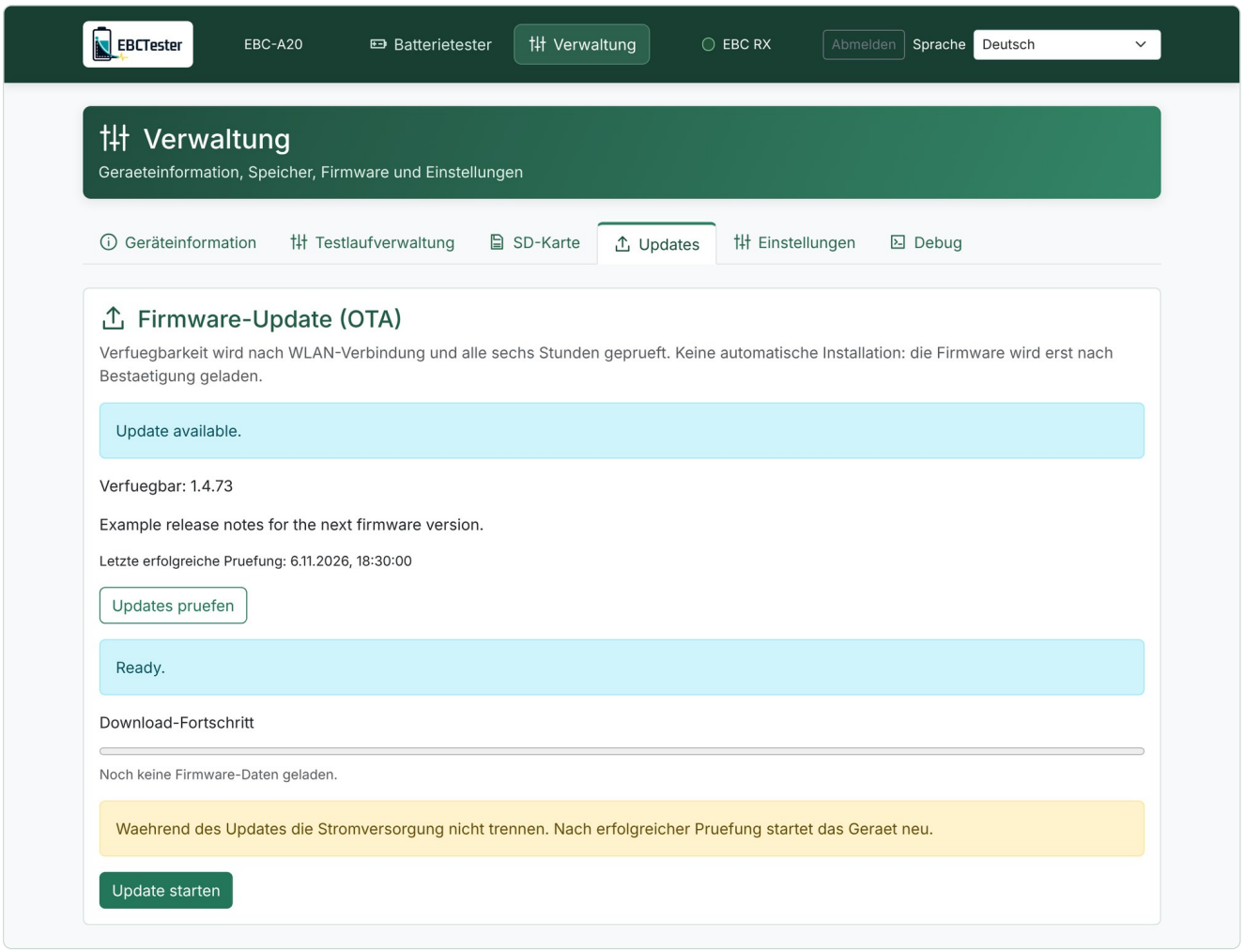


Abbildung 16: Firmware-Update

Ist das Gerät mit dem Internet verbunden, sucht es nach dem Verbinden, alle sechs Stunden und bei jedem Öffnen der Weboberfläche nach neuer Firmware. Dabei wird nur eine kleine Beschreibungsdatei geladen – **Firmware wird nie automatisch installiert.**

1. Ist ein Update verfügbar, werden die neue Version und ihre Versionshinweise angezeigt. **Updates prüfen** wiederholt die Prüfung von Hand.
2. Sicherstellen, dass kein Test läuft, dann auf **Update starten** klicken und die Version bestätigen.
3. Die Firmware wird in den inaktiven Speicherbereich geladen und geprüft (Größe und SHA-256-Prüfsumme). Der Fortschritt wird angezeigt.
4. Nach erfolgreicher Prüfung startet das Gerät nach drei Sekunden neu. Die neue Firmware wird erst dauerhaft übernommen, wenn sie 10 Sekunden stabil läuft; andernfalls wird automatisch die vorherige Version wiederhergestellt.

> **Warnung**
> Während eines Updates nie die Stromversorgung trennen und nie während eines Akkutests aktualisieren.

4.5 Einstellungen

Der Reiter **Einstellungen** fasst alle Geräteeinstellungen zusammen. Beim Speichern einer Einstellung wird als zusätzlicher Schutz das aktuelle **Anmeldepasswort** abgefragt (im Dialog mit **Admin-Passwort** bezeichnet) – auch dann, wenn die Browser-Anmeldung abgeschaltet ist. Bei Abbruch oder falschem Passwort wird nichts gespeichert. Das Passwort des Zugangspunkts wird hier nicht verwendet.

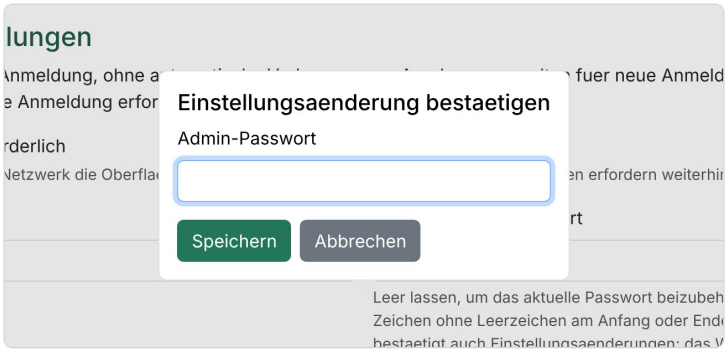


Abbildung 17: Passwortabfrage beim Speichern von Einstellungen

Anmeldeeinstellungen

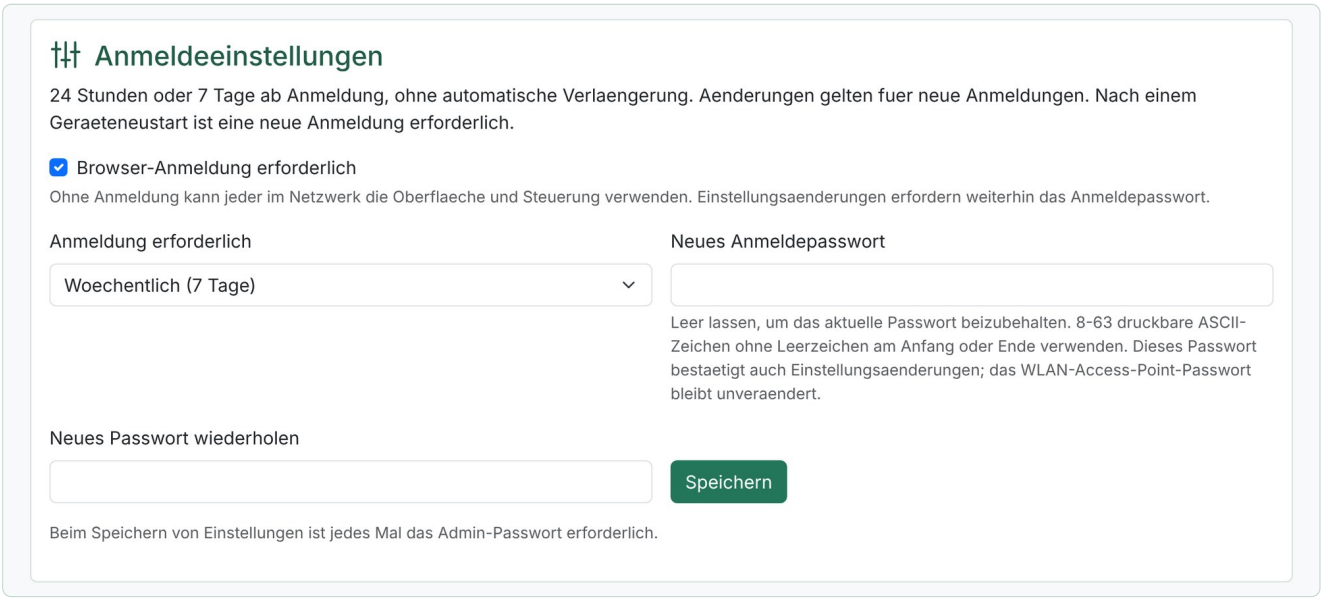


Abbildung 18: Anmeldeeinstellungen

Einstellung	Beschreibung
Browser-Anmeldung erforderlich	Standardmäßig ein. Ist sie aus, öffnet sich die Weboberfläche ohne Anmeldung, und der Knopf **Abmelden** entfällt.
Anmeldung erforderlich	**Täglich (24 Stunden)** (Standard) oder **Wöchentlich (7 Tage)**. Die Dauer zählt ab der Anmeldung, verlängert sich nicht durch Benutzung und gilt für neue Anmeldungen. Nur verfügbar, solange die Browser-Anmeldung eingeschaltet ist.
Neues Anmeldepasswort / Neues Passwort wiederholen	Das neue Passwort zweimal eingeben. 8–63 druckbare ASCII-Zeichen (Buchstaben, Ziffern und gängige Sonderzeichen, keine Umlaute), keine Leerzeichen am Anfang oder Ende. Beide Felder leer lassen, um das aktuelle Passwort zu behalten.

Auf **Speichern** klicken und mit dem *aktuellen* Anmeldepasswort bestätigen – auch dann, wenn Sie gerade das Passwort selbst ändern. Das neue Passwort wird dauerhaft auf dem Gerät gespeichert und übersteht Neustarts und Firmware-Updates. Das Passwort des Zugangspunkts ändert sich dabei nicht.

Eine Passwortänderung oder das Ein- und Ausschalten der Browser-Anmeldung beendet alle bestehenden Sitzungen; die Seite lädt automatisch neu. Melden Sie sich danach mit dem neuen Passwort an.

> **Warnung**
> Bei abgeschalteter Browser-Anmeldung kann **jeder im Netzwerk** alle Daten sehen und den Tester bedienen – einschließlich Starten und Stoppen von Tests. Nutzen Sie das nur in einem Netz, dem Sie vollständig vertrauen. Einstellungsänderungen verlangen weiterhin das Anmeldepasswort.

WLAN einrichten



Abbildung 19: WLAN-Einstellungen

Netzwerkname (SSID) und Passwort Ihres Heimnetzes. Für ein offenes Netz das Passwort leer lassen; sonst 8–63 Zeichen oder 64 Hexadezimalzeichen. Die Zugangsdaten werden auf dem Gerät gespeichert.

EBC tester

Auswahl des Testermodells – siehe Testermodell auswählen.

Home Assistant MQTT

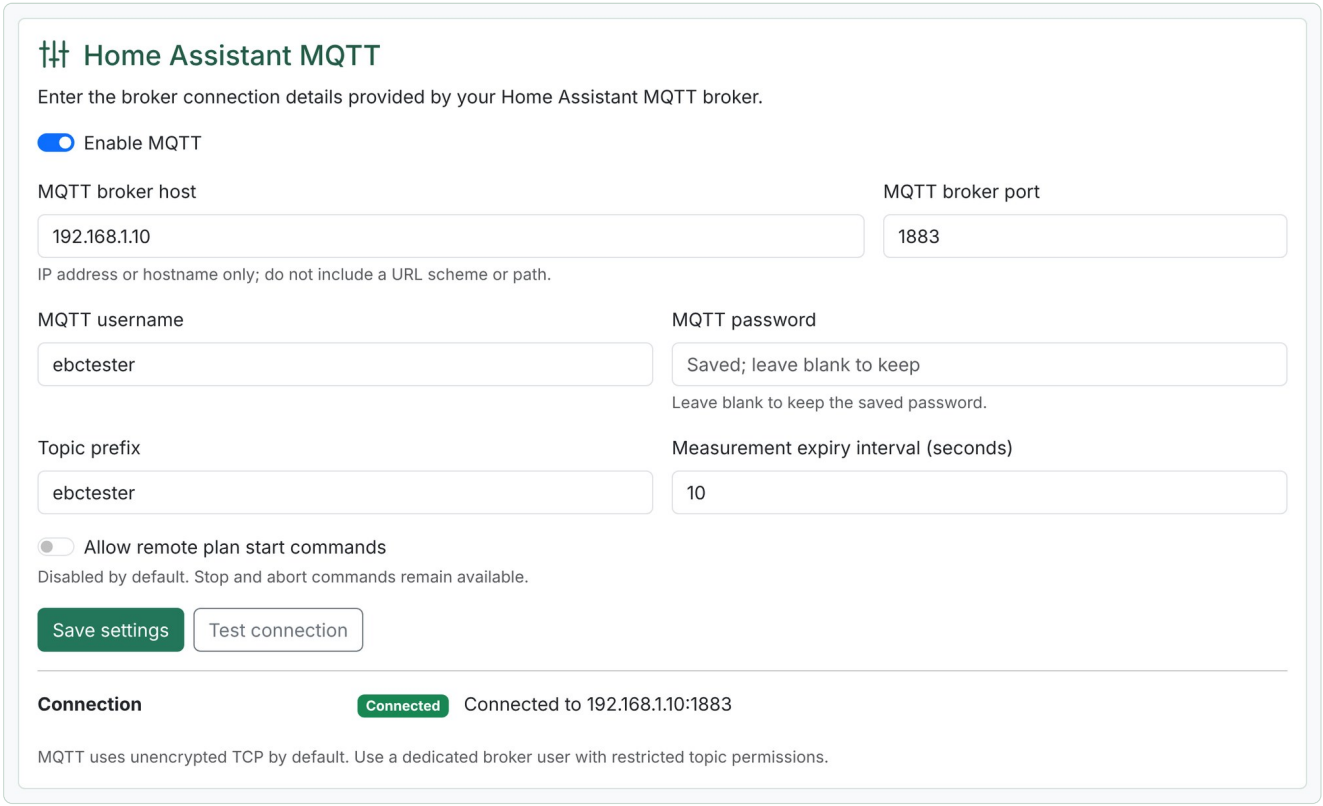


Abbildung 20: Home Assistant MQTT (nur vorhanden, wenn in die Firmware eingebaut)

Optionale Verbindung zu einem MQTT-Broker, üblicherweise dem von Home Assistant. Das Gerät erscheint dann automatisch in Home Assistant (MQTT Discovery) und veröffentlicht Spannung, Strom und Kapazität jedes aufgezeichneten Messwerts. Die Karte ist in Firmware 1.4.72 noch englisch beschriftet.

Enable MQTT	Schaltet den Client ein oder aus.
Broker host / port	IP-Adresse oder Hostname (ohne `mqtt://`), Port meist 1883.
User name / password	Zugang zum Broker. Das Passwort leer lassen, um das gespeicherte zu behalten.
Topic prefix	Anfang aller Topics, z. B. `ebctester`.
Measurement expiry interval (s)	Abstand der aufgezeichneten Messwerte; nach drei Intervallen ohne neue Daten werden die Werte „nicht verfügbar“.
Allow remote plan start commands	Standardmäßig aus. Nur wenn eingeschaltet, lässt sich ein Plan über MQTT starten. Stopp und Abbrechen funktionieren immer.

Save settings übernimmt die Einstellungen sofort, **Test connection** verbindet sich mit den gespeicherten Einstellungen neu. Darunter steht der Verbindungszustand. MQTT ist unverschlüsselt – verwenden Sie einen eigenen Broker-Benutzer mit eingeschränkten Rechten. Einzelheiten in Abschnitt 12.2.

MariaDB-Sicherung

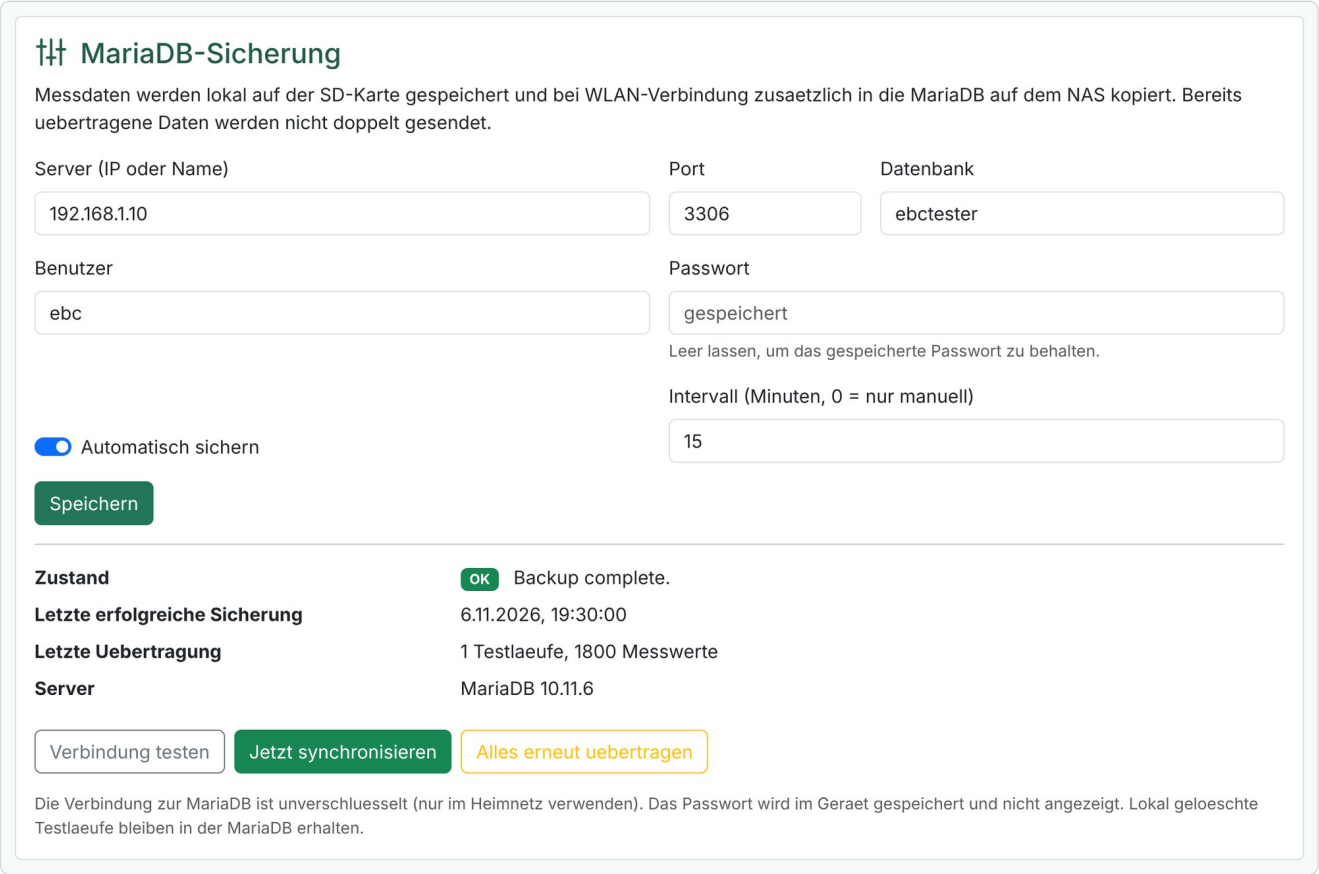


Abbildung 21: MariaDB-Sicherung

Kopiert die aufgezeichneten Testläufe in eine MariaDB-Datenbank, z. B. auf einem NAS. Übertragen werden nur neue Daten; wiederholte oder unterbrochene Synchronisationen erzeugen keine Duplikate.

- Server, Port (3306), Datenbank, Benutzer und Passwort eingeben; **Automatisch sichern** und ein Intervall in Minuten wählen (0 = nur manuell). Speichern.
- **Verbindung testen** prüft den Zugang, **Jetzt synchronisieren** startet sofort eine Übertragung, **Alles erneut übertragen** sendet alle Daten noch einmal (ohne Duplikate).
- Der Statusbereich zeigt Zustand, letzte erfolgreiche Sicherung, Anzahl übertragener Läufe und Messwerte sowie die Serverversion.

Die automatische Sicherung läuft nur mit WLAN, bereiter SD-Karte und gespeichertem Passwort. Auf dem Gerät gelöschte Läufe bleiben in MariaDB erhalten. Die Datenbanktabellen müssen auf dem Server bereits existieren – siehe Abschnitt 11.3. Die Verbindung ist unverschlüsselt: nur im Heimnetz verwenden.

Debug-Protokoll

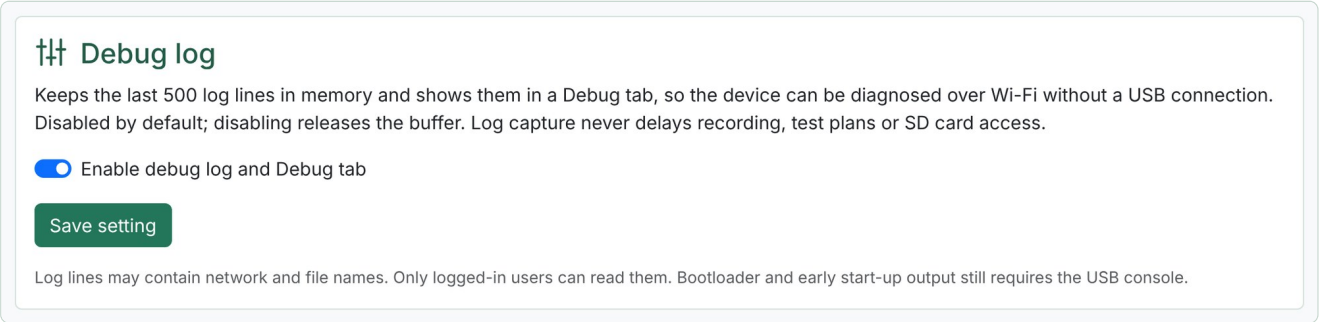


Abbildung 22: Einstellung Debug-Protokoll

Wenn eingeschaltet, hält das Gerät die letzten 500 Protokollzeilen im Speicher und zeigt sie im Reiter **Debug**. Standardmäßig aus; Ausschalten gibt den Speicher frei. Die Protokollierung verzögert nie Aufzeichnung, Pläne oder SD-Zugriffe. Die Karte ist in Firmware 1.4.72 noch englisch beschriftet.

Sprache



Abbildung 23: Sprache

Englisch und Deutsch sind eingebaut. Weitere Sprachen lassen sich als JSON-Dateien im Ordner `/languages` der SD-Karte hinzufügen (siehe Anhang A). **Sprachen neu laden** liest den Ordner erneut ein; danach die Sprache in der Kopfzeile wählen.

4.6 Debug

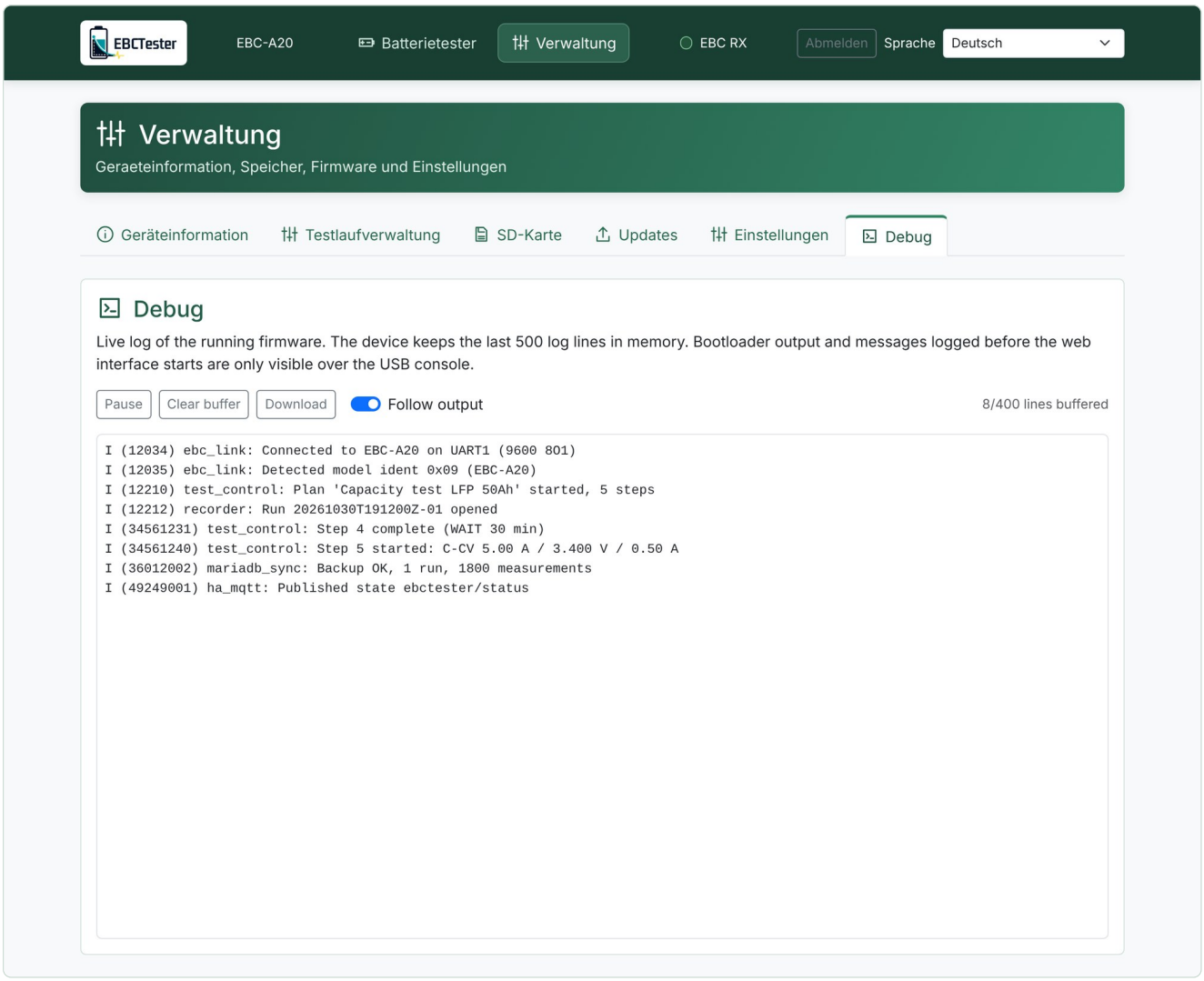


Abbildung 24: Reiter Debug

Live-Protokoll der laufenden Firmware – hilfreich, wenn etwas nicht funktioniert und kein USB-Kabel zur Hand ist. **Pause** hält die Anzeige an, **Clear buffer** leert sie, **Download** speichert die Zeilen als Textdatei, und **Follow output** hält die Ansicht auf der neuesten Zeile. Protokollzeilen können Netzwerk- und Dateinamen enthalten, aber nie Passwörter. Meldungen aus der allerersten Startphase sind nur auf der USB-Konsole sichtbar.

5 Typische Abläufe

5.1 Kapazität eines Akkus messen

1. **Batterie:** Batteriedaten eingeben und auf **Übernehmen und in Datenbank speichern** klicken (oder eine gespeicherte Batterie laden und auf **Übernehmen** klicken).
2. **Plan:** unter **Automatik** einen Plan laden oder anlegen: Laden (C-CV) – Warten – Entladen (D-CC) – Warten – optional auf Lagerspannung laden.
3. **Verbinden:** unter **Manuell** auf **Verbinden** klicken (oder automatisches Verbinden einschalten). Die RX-Anzeige in der Kopfzeile blinkt.
4. **Starten:** unter **Automatik** auf **Plan starten** klicken. Den Browser können Sie jetzt schließen.
5. **Verfolgen:** jederzeit **Messwerte** öffnen. Die Kapazität des Entladesegments in der Segment-Timeline ist die gemessene Kapazität.
6. **Abschließen:** Ist der Plan fertig, wechselt der Status des Laufs auf *Abgeschlossen*. Bei Bedarf dem Lauf in der **Testlaufverwaltung** einen aussagekräftigen Namen geben.

5.2 Innenwiderstand bei 50 % Ladezustand messen

1. Batterie wie oben übernehmen und mit Vierleitermessung anschließen.
2. Unter **Innenwiderstand** die Option **Automatische Vorbereitung und Messung** wählen, die nutzbare Kapazität (am besten das Ergebnis eines Kapazitätstests), die Ladeparameter und *Ziel-Ladezustand = 50 %* eintragen.
3. Den Pulsstrom festlegen (z. B. 0,2 C bis 1 C, innerhalb der Grenzen des Akkus), die Schutzgrenzen prüfen, speichern, die Bestätigung ankreuzen und starten.
4. Nach Abschluss die drei Ergebnisse vergleichen. Um Akkus zu vergleichen, unter gleichen Bedingungen messen (Temperatur, Ladezustand, Ruhezeit).

5.3 Daten sichern

- **Einfach:** `ebc_a20.db` unter **SD-Karte** herunterladen, wenn kein Test läuft. Die Datei lässt sich mit jedem SQLite-Werkzeug öffnen (z. B. „DB Browser for SQLite“).
- **Automatisch:** die MariaDB-Sicherung einrichten. Läufe auf der SD-Karte erst löschen, wenn sicher ist, dass sie übertragen wurden.

6 Fehlersuche

Problem	Mögliche Ursache	Abhilfe
Zugangspunkt `BatteryTester-…` nicht sichtbar	Keine Antenne, keine Versorgung oder Gerät startet noch	Antenne und Versorgung prüfen (leuchtet D6?); nach dem Einschalten 10–20 s warten.
AP-Passwort unbekannt	Beim ersten Start zufällig erzeugt	Im Startprotokoll über USB-C ablesen (serielles Terminal, RESET drücken).
Anmeldepasswort wird nicht akzeptiert	Passwort wurde geändert, oder der Anfangswert ist noch aktiv	`ebctester` versuchen (gilt, bis ein Passwort gespeichert ist). Ein vergessenes gespeichertes Passwort lässt sich nur durch Löschen des Flash-Speichers und Neuinstallation der Firmware zurücksetzen – dabei gehen alle Geräteeinstellungen verloren, die Daten auf der SD-Karte bleiben erhalten.
Seite 192.168.4.1 lädt nicht	Handy ist zurück auf mobile Daten oder ein anderes Netz gewechselt	Mobile Daten abschalten, solange das Handy mit dem AP verbunden ist; WLAN-Verbindung prüfen.
Anmeldung führt immer wieder zur Anmeldeseite	Sitzung abgelaufen oder Gerät neu gestartet; Browser blockiert Cookies	Neu anmelden; Cookies für die Geräteadresse erlauben.
Anmeldung abgelehnt, obwohl das Passwort stimmt	Höchstzahl von 8 gleichzeitigen Sitzungen erreicht	In anderen Browsern abmelden oder das Gerät neu starten.
RX-Anzeige bleibt grau	Tester nicht verbunden oder falsch verdrahtet	Auf **Verbinden** klicken; Kreuzung von TX/RX und GND prüfen; prüfen, ob der Tester eingeschaltet ist.
Fehler „Tester passt nicht“	Gewähltes Modell weicht vom angeschlossenen ab	Unter Einstellungen → EBC tester das richtige Modell wählen, dann Verbinden.
Startknöpfe gesperrt	Siehe gelben Hinweis unter dem Knopf	Typisch: nicht verbunden, warte auf erste Meldung, Plan läuft, Befehl wartet auf Bestätigung.
„Datenbank nicht bereit“	Keine Karte, Karte nicht FAT32, voll oder defekt	FAT32-Karte einsetzen; Reiter SD-Karte prüfen. exFAT-Karten am PC auf FAT32 umformatieren.
Zeiten als Laufzeit statt Datum	Uhr nicht synchronisiert (kein Internet)	Das Gerät mit einem WLAN mit Internetzugang verbinden.
Plan zeigt „Unterbrochen“	Stromausfall oder Neustart während des Plans	Akku prüfen, auf **Status zurücksetzen** klicken, Plan bei Bedarf neu starten.
DCIR-Ergebnis „ungültig“	Messpunkt zu spät, Spannungsabfall zu klein, kein Strom	Pulsstrom erhöhen, Vierleitermessung und Zeittoleranz prüfen.
Update-Prüfung schlägt fehl	Kein Internet, Uhr noch nicht synchronisiert, Server nicht erreichbar	Nach dem Verbinden einige Minuten warten; mit **Updates prüfen** wiederholen.
Einstellungen werden nicht gespeichert	Anmeldepasswort falsch oder Dialog abgebrochen	Im Bestätigungsdialog das aktuelle Anmeldepasswort eingeben (nicht das Passwort des Zugangspunkts).
Sprachdatei wird nicht angeboten	Falscher Ordner, ungültiges JSON oder falscher Code	In `/languages` ablegen, die Warnung in der Kopfzeile beachten, auf Sprachen neu laden klicken.

Bleibt das Problem bestehen, das Debug-Protokoll einschalten, das Problem nachstellen und das Protokoll herunterladen. Wenn Sie ein Problem auf GitHub melden, geben Sie Platine, Firmware-Version, die Schritte zum Nachstellen und das Protokoll an – aber nie Passwörter oder private Daten.

Teil II – Technische Referenz

Dieser Teil richtet sich an alle, die die Hardware selbst aufbauen, die Firmware kompilieren, EBCTester in andere Systeme einbinden oder seine Funktionsweise verstehen möchten. Er behandelt Schaltung, Anschlüsse und Bauteile, Firmware-Konfiguration, das Testerprotokoll, die Datenspeicherung, Schnittstellen und Sicherheit.

7 Systemarchitektur

7.1 Aufbau der Firmware

Die Firmware ist in C/C++ mit Espressifs ESP-IDF 6.0.1 für den ESP32-S3 geschrieben. Sie ist in funktionale Komponenten gegliedert; das Hauptprogramm initialisiert nur die Dienste. Netzwerk, Updates und SD-Kartenverwaltung sind eigene, wiederverwendbare Repositories, die als Git-Submodule eingebunden sind.

Repository / Komponente	Aufgabe
EBCTester	Anwendung: Testersteuerung, Pläne, Aufzeichnung, Diagramme, DCIR, MariaDB-Sicherung
`ebc_link`	UARTs, Rahmenparser, Modellerkennung und Kommunikation mit dem Tester
`test_control` / `test_store`	Manuelle Steuerung, Testpläne, Batterien, Aufzeichnung von Testläufen
`resistance_test`	DCIR-Vorbereitung, Lastpuls und Auswertung
`sqlite3` / `battery_database`	SQLite-Adapter, SD-Dateisystemschicht und Datenbankschema
`battery_dashboard`	Reiter Messwerte, Diagramme (Chart.js) und Testlaufverwaltung
`mariadb_sync`	Inkrementelle Sicherung nach MariaDB
`ha_mqtt`	Optionale Home-Assistant-MQTT-Anbindung
`ui_localization` / `debug_log`	Zusätzliche Sprachen von SD; Protokoll im Speicher und Reiter Debug
ota-suite	WLAN-Einrichtung, Zugangspunkt, Webserver und gemeinsame Oberfläche, Anmeldung, manuelle OTA-Updates, Status-LED
sd-card-manager	SDMMC/SPI-Treiber, Kartenerkennung, gemeinsame Speichersperre, Reiter Dateiverwaltung

7.2 Datenfluss

Der Tester sendet etwa einmal pro Sekunde eine Statusmeldung. `ebc_link` prüft jede Meldung (Prüfsumme, Modellkennung) und reicht sie an die Steuerung weiter. Während ein Test aktiv ist, mittelt `test_store` eine einstellbare Zahl von Meldungen (Standard 20, also etwa alle 20 Sekunden ein gespeicherter Messwert) und schreibt den Messwert in einer Datenbanktransaktion auf die SD-Karte. Pläne und DCIR-Tests sind Zustandsautomaten in eigenen Tasks; sie reagieren auf die empfangenen Meldungen und senden Befehle an den Tester. Die Weboberfläche liest nur Daten und sendet Befehle über authentifizierte HTTP-Anfragen – für den Fortgang eines Tests wird sie nie benötigt.

8 Hardware

Die EBCTester-Platine (Revision 1) ist eine kompakte Steuerplatine. Sie enthält keine eigene Messschaltung – die gesamte Mess- und Leistungselektronik steckt im ZKE-Tester. Die Platine stellt den Mikrocontroller, die Stromversorgung, die SD-Karte, die Echtzeituhr und eine pegelgewandelte serielle Schnittstelle bereit.

> **Hinweis**
> Stand der Hardware-Dokumentation: Schaltplan und Platinenlayout der Revision 1 vom 4. November 2026, Stückliste vom 4. November 2026. Die Platine ist noch nicht gefertigt und in Betrieb genommen; kleine Änderungen sind daher noch möglich. Neuere Revisionen finden Sie im GitHub-Projekt.

8.1 Blockschaltbild

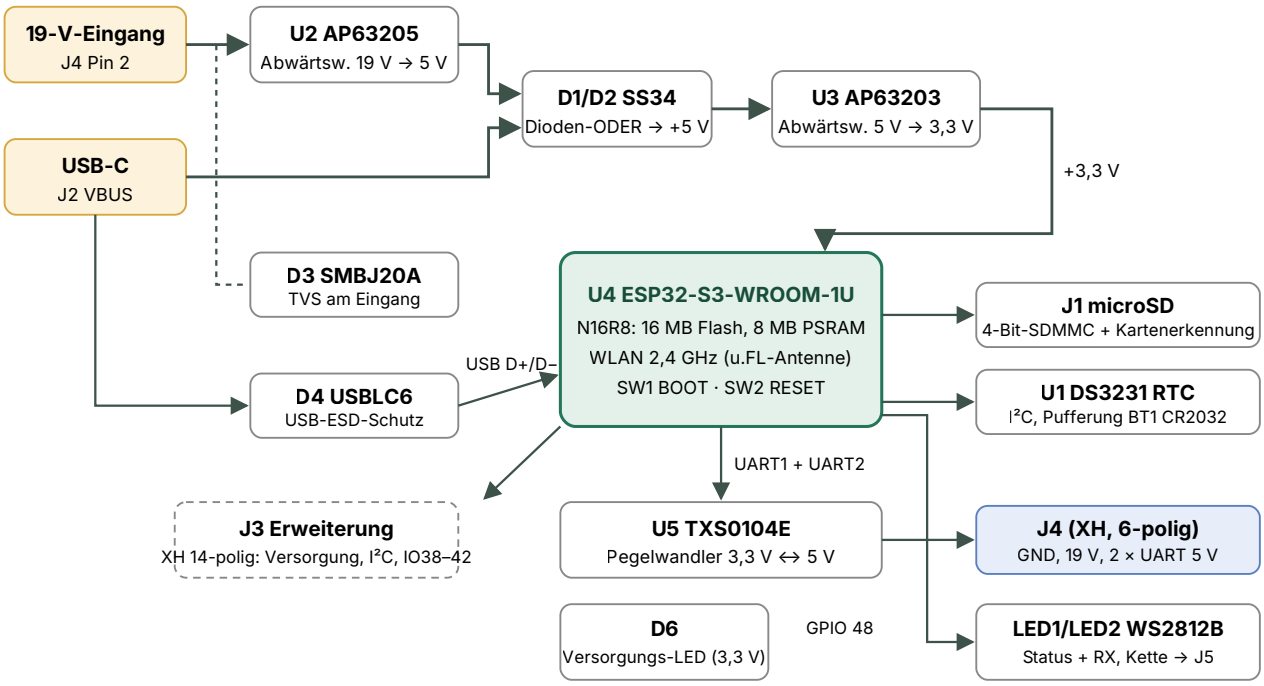


Abbildung 25: Blockschaltbild der EBCTester-Platine

8.2 Schaltungsbeschreibung

Stromversorgung

Der 19-V-Eingang (vom Netzteil des EBC-A20 über J4) ist durch die unidirektionale TVS-Diode D3 geschützt (SMBJ20A, Kathode an +19 V; Sperrspannung 20 V, Klemmspannung 32,4 V – unter der absoluten Höchstspannung von 35 V am Eingang von U2). D3 schützt durch Leiten auch gegen Verpolung; die Versorgung sollte deshalb strombegrenzt oder abgesichert sein. Der Synchron-Abwärtswandler U2 (AP63205, fest 5 V, bis 2 A) erzeugt mit L1 (4,7 µH) 5 V. Dieser Ausgang und die USB-Versorgung (VBUS) werden über die Schottky-Dioden D1 und D2 (SS34) zur **+5-V-Schiene** zusammengeführt, die nach dem Diodenabfall etwa 4,6–4,7 V führt. Die Platine läuft also am 19-V-Eingang, an USB oder an beiden gleichzeitig. Mit USB allein ist der Gesamtstrom auf die 500 mA des USB-Anschlusses begrenzt.

Ein zweiter Abwärtswandler U3 (AP63203, fest 3,3 V) mit L2 versorgt aus der +5-V-Schiene den ESP32-S3 und die gesamte Logik; 3,3 V stehen also aus beiden Quellen zur Verfügung. Ein- und Ausgangskondensatoren folgen dem Datenblatt von Diodes Inc.: 10 µF/50 V auf der 19-V-Seite, 2 × 22 µF an jedem Ausgang, 100-nF-Bootstrap-Kondensatoren. Das Datenblatt empfiehlt für die 3,3-V-Version 3,9 µH; 4,7 µH liegen im zulässigen Bereich (2,2–10 µH).

Mikrocontroller und USB

U4 ist ein Modul ESP32-S3-WROOM-1U-N16R8 mit 16 MB Flash und 8 MB Octal-PSRAM. Die Variante „U“ hat keine Leiterplattenantenne, sondern eine u.FL-Buchse – eine externe 2,4-GHz-Antenne ist zwingend. GPIO 35–37 werden intern vom Octal-PSRAM belegt und dürfen nicht verwendet werden. Die USB-C-Buchse J2 ist mit dem nativen USB des ESP32-S3 (GPIO 19/20) verbunden – zum Flashen und für die Konsole; D4 (USBLC6-2SC6) schützt die Datenleitungen vor ESD. R10/R11 (5,1 kΩ an CC1/CC2) melden die Platine an USB-C-Netzteilen als USB-Gerät an.

SW1 (BOOT, GPIO 0, Pull-up R12) und SW2 (RESET, EN, Pull-up R13 mit C16 für einen definierten Einschalt-Reset) ermöglichen den Download-Modus von Hand: **BOOT halten, RESET drücken und loslassen, BOOT loslassen.**

microSD-Karte

J1 (Hirose DM3AT) ist im 4-Bit-SD-Modus angeschlossen. Alle Daten- und Befehlsleitungen haben, wie von der SD-Spezifikation gefordert, 10-kΩ-Pull-ups (R1–R5). Der Kartenerkennungsschalter ist im Ruhezustand offen und schließt beim Einstecken einer Karte; mit dem Pull-up R6 ist das Signal `SD_CD` **low, wenn eine Karte steckt**.

Echtzeituhr

U1 (DS3231SN) ist eine temperaturkompensierte Echtzeituhr (±2 ppm) am I²C-Bus mit 4,7-kΩ-Pull-ups (R7/R8). Ihr Ausgang INT/SQW (Open Drain, Pull-up R9) führt zu GPIO 14 und kann als Interrupt oder Weckquelle aus dem Tiefschlaf dienen. Eine nicht wiederaufladbare Zelle CR2032 (BT1) an VBAT hält die Uhrzeit, wenn die Platine stromlos ist. C2 blockt den Versorgungspin VCC ab, C1 den Pufferanschluss VBAT; die Zellenspannung lässt sich an TP11 messen. *Firmware 1.4.72 nutzt die Echtzeituhr noch nicht*; die Uhrzeit wird per SNTP bezogen, sobald WLAN mit Internetzugang verfügbar ist.

Serielle Schnittstelle

Der Tester arbeitet mit 5-V-TTL-Pegeln, die Eingänge des ESP32-S3 sind aber nicht 5-V-tolerant. U5 (TXS0104E) ist ein bidirektionaler 4-Bit-Pegelwandler mit automatischer Richtungserkennung zwischen 3,3 V (VCCA) und der +5-V-Schiene (VCCB). Er führt UART1 (Tester) und UART2 (reserviert für den PC-Dongle) zum Stecker J4. Der TXS0104E eignet sich für Gegentaktsignale bei der verwendeten niedrigen Baudrate (9600 Bd).

Status-LEDs

LED1 und LED2 sind RGB-LEDs WS2812B mit eingebautem Treiber. Sie sind als Kette über den Vorwiderstand R15 (470 Ω) an GPIO 48 angeschlossen. LED1 zeigt den Betriebszustand, LED2 den Empfang von Testermeldungen (siehe Abschnitt 2.5). Jede LED hat einen 100-nF-Abblockkondensator (C17, C18). Die WS2812B laufen an der +5-V-Schiene, der ESP32 treibt die Datenleitung aber mit 3,3 V. Damit dieser Pegel über der Eingangsschwelle für High von LED1 ($0{,}7 \times V_{DD}$) bleibt, wird LED1 über die Schottky-Diode D5 versorgt, die ihre Versorgung um etwa 0,3 V absenkt. LED2 erhält von LED1 das aufbereitete 5-V-Signal. Der Datenausgang von LED2 ist über R16 (470 Ω) auf den Stecker J5 geführt, so dass sich weitere WS2812B-LEDs an die Kette anhängen lassen.

Versorgungs-LED

D6 (gelbgrün, 0603) liegt über R14 (470 Ω) an +3,3 V und leuchtet, sobald die 3,3-V-Versorgung anliegt – unabhängig von der Firmware.

8.3 Anschlüsse, Lötbrücke und Messpunkte

J4 – Testeranschluss (JST XH, 6-polig, 2,5 mm)

Pin	Signal	Beschreibung
1	GND	Masse
2	VIN_19V	19-V-Versorgungseingang (vom Netzteil des EBC-A20)
3	UART1_TX_5V	Daten von EBCTester zum Tester → RX des Testers
4	UART1_RX_5V	Daten vom Tester → TX des Testers
5	UART2_TX_5V	Reserviert für die PC-Dongle-Brücke
6	UART2_RX_5V	Reserviert für die PC-Dongle-Brücke

TX/RX sind aus Sicht des ESP32 benannt. UART2 wird von der Firmware initialisiert, die Brücke zwischen der PC-Software des Herstellers und dem Tester ist aber noch nicht umgesetzt.

J3 – Erweiterungsstecker (JST XH, 14-polig, 2,5 mm)

Pin	Signal	Pin	Signal
1	VIN_19V	8	GPIO 42
2	+3,3 V	9	GPIO 41
3	+5 V	10	GPIO 40
4	VBUS	11	GPIO 39
5	RTC_INT (GPIO 14)	12	GPIO 38
6	I2C_SDA (GPIO 15)	13	GND
7	I2C_SCL (GPIO 16)	14	GND

J3 führt die Versorgungsschienen, den I²C-Bus mit dem Interrupt der Echtzeituhr und fünf freie GPIOs für Erweiterungen (z. B. Display, Temperatursensoren). Die aktuelle Firmware nutzt ihn nicht. Die 3,3-V-GPIOs sind nicht 5-V-tolerant.

J5 – LED-Kettenerweiterung (JST XH, 3-polig, 2,5 mm)

Pin	Signal	Beschreibung
1	+5 V	Versorgung für externe WS2812B-LEDs (aus der +5-V-Schiene, ca. 4,6–4,7 V)
2	DATA	Datenausgang von LED2 über R16 (470 Ω)
3	GND	Masse

Externe LEDs beziehen ihren Strom aus der +5-V-Schiene. Bei reiner USB-Versorgung muss der Gesamtstrom innerhalb der 500 mA des USB-Anschlusses bleiben (eine WS2812B zieht bei vollem Weiß bis zu ca. 60 mA).

JP1 – Lötbrücke „Custom_Settings“ (GPIO 47)

Eine Lötbrücke mit drei Pads, die GPIO 47 auf +3,3 V oder GND legt. Sie ist für eine Hardware-Konfigurationsoption reserviert (z. B. eine Platinenvariante oder eine beim Start gelesene Voreinstellung) und wird von Firmware 1.4.72 nicht ausgewertet. Im Auslieferungszustand sind die Pads 1 und 2 verbunden (Footprint „Bridged12“), GPIO 47 liest also High; für Low die Brücke auftrennen und die Pads 2–3 schließen. GPIO 47 ist kein Strapping-Pin, beide Stellungen sind also für den Start unkritisch. Ist JP1 offen, sollte die Firmware einen internen Pull-Widerstand einschalten, damit der Eingang nicht offen schwebt.

Messpunkte

TP1–TP11 sind runde SMD-Pads (1,5 mm Durchmesser) für Prüfspitzen; es sind keine eigenen Bauteile. Neben jedem Messpunkt steht im Bestückungsdruck der Signalname (z. B. *19V*, *+5V*, *+4,6V-4,7V*, *VBAT_RTC*).

Messpunkt	Signal	Erwarteter Wert
TP3	19-V-Eingang	ca. 19 V (bei Versorgung über J4)
TP4	5-V-Wandlerausgang (vor der Diode)	5,0 V
TP1	+5-V-Schiene	4,6 – 4,7 V
TP6	USB VBUS	5 V bei angeschlossenem USB
TP2	+3,3 V	3,3 V
TP5	GND	Bezugspunkt
TP7 / TP8	BOOT (GPIO 0) / RESET (EN)	3,3 V in Ruhe, 0 V gedrückt
TP9 / TP10	RXD0 (GPIO 44) / TXD0 (GPIO 43)	UART0: Startmeldungen des ESP32 (3,3-V-Pegel, 115200 Bd)
TP11	VBAT_RTC (CR2032)	ca. 3,0 V bei eingesetzter Zelle

GPIO-Belegung der Platinenrevision 1

Funktion	GPIO	Funktion	GPIO
UART1 TX / RX (Tester)	10 / 11	SD CLK / CMD	6 / 7
UART2 TX / RX (Dongle)	8 / 9	SD D0 / D1 / D2 / D3	5 / 4 / 18 / 17
I²C SDA / SCL (RTC)	15 / 16	SD-Kartenerkennung (low = Karte)	13
RTC INT/SQW	14	Daten LED1/LED2 (WS2812B)	48
JP1 Custom_Settings	47	Strapping-Pins, frei lassen	3, 45, 46
USB D− / D+	19 / 20	Erweiterung J3	38 – 42
BOOT	0	PSRAM (nicht verwenden)	35 – 37
UART0 RXD0 / TXD0 (TP9 / TP10)	44 / 43	Nicht angeschlossen (No-Connect)	1, 2, 12, 21

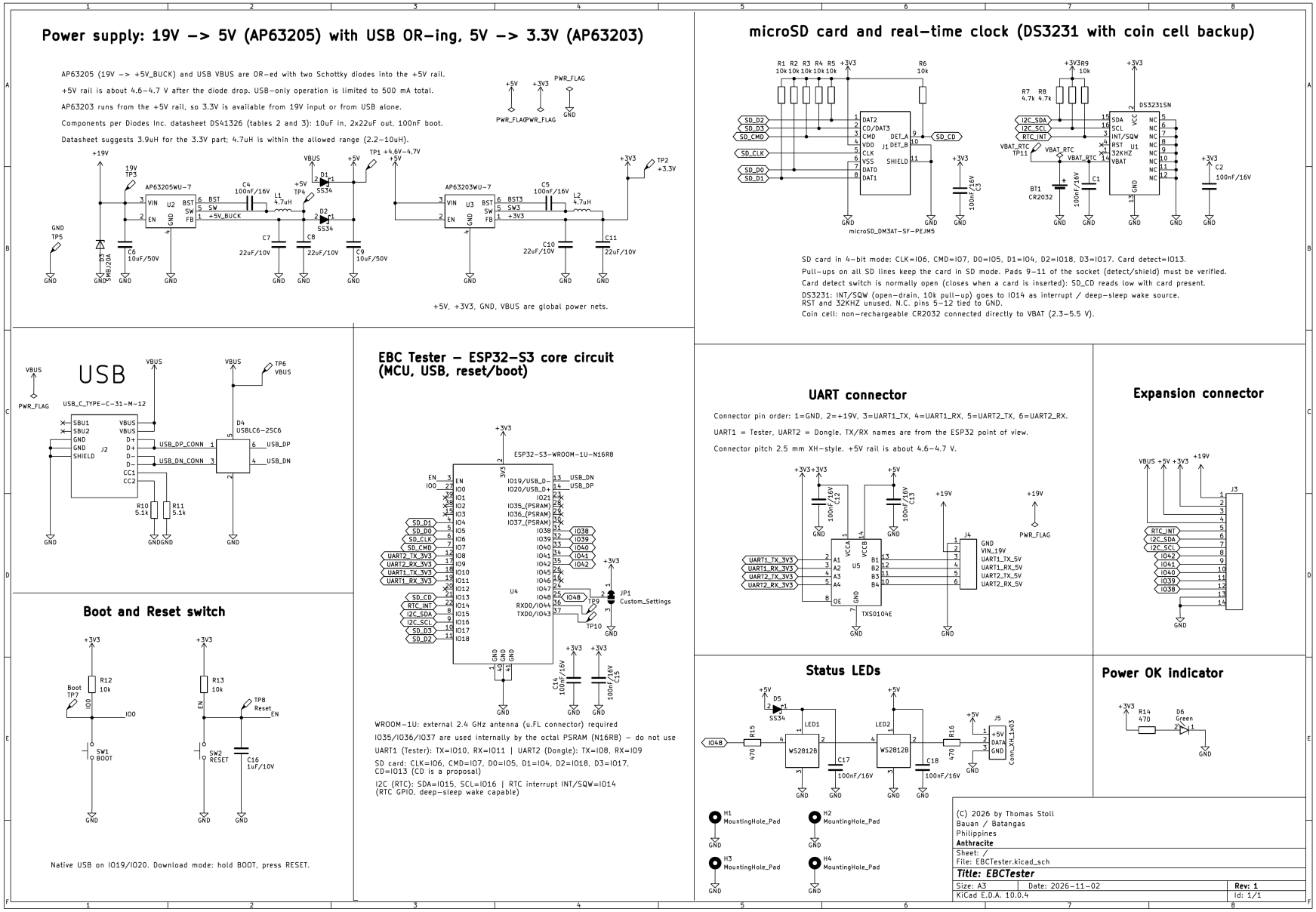


Abbildung 26: Schaltplan der EBCTester-Platine, Revision 1 (KiCad 10). Das PDF liegt im Repository unter docs/EBCTester_Schematic.pdf.

8.4 Stückliste

26 Positionen, 57 Bauteile (53 SMD, 4 THT). Alle Teile wurden zwischen dem 1. und 4. November 2026 bei LCSC geprüft; prüfen Sie vor der Bestellung den Lagerbestand. Fertige BOM-Dateien zum Hochladen bei JLCPCB und PCBWay liegen im Hardware-Repository.

Bez.	Anz.	Wert / Funktion	Gehäuse	Hersteller / Teilenummer	LCSC
U1	1	Echtzeituhr mit TCXO, I²C	SOIC-16W	Analog Devices DS3231SN#T&R	C9866
U2	1	Abwärtswandler 19 V → 5 V, 2 A	TSOT-23-6	Diodes AP63205WU-7	C2071056
U3	1	Abwärtswandler 5 V → 3,3 V, 2 A	TSOT-23-6	Diodes AP63203WU-7	C780769
U4	1	ESP32-S3-Modul, 16 MB/8 MB, u.FL	Modul	Espressif ESP32-S3-WROOM-1U-N16R8	C3013946
U5	1	Pegelwandler 3,3 V / 5 V	TSSOP-14	TI TXS0104EPWR	C44955
LED1, LED2	2	RGB-LED mit Treiber, Status / RX	5050	Worldsemi WS2812B-B/W	C114586
J1	1	microSD-Halter, Kartenerkennung	SMD	Hirose DM3AT-SF-PEJM5	C114218
J2	1	USB-C-Buchse (USB 2.0)	SMD	HRO TYPE-C-31-M-12	C165948
J3	1	Stiftleiste 14-polig, 2,5 mm	THT	BOOMELE XH-14A	C20200
J4	1	Stiftleiste 6-polig, 2,5 mm	THT	BOOMELE XH-6A	C5663
J5	1	Stiftleiste 3-polig, 2,5 mm (LED-Kette)	THT	BOOMELE XH-3A	C2316
D1, D2, D5	3	Schottky 40 V / 3 A (ODER-Schaltung; Versorgung LED1)	SMA	MDD SS34	C8678
D3	1	TVS 20 V, 600 W	SMB	TWGMC SMBJ20A	C726828
D4	1	USB-ESD-Schutz	SOT-23-6	ST USBLC6-2SC6	C7519
D6	1	LED gelbgrün, Versorgung OK	0603	XINGLIGHT XL-1608SYGC-04	C965806
L1, L2	2	Speicherdrossel 4,7 µH	6 × 6 mm	Bourns SRN6045TA-4R7M	C2044594
SW1, SW2	2	Taster BOOT / RESET	5,1 × 5,1 mm	XKB TS-1187A-B-A-B	C318884
BT1	1	Halter CR2032	THT	MYOUNG BS-07-A1BJ001	C2979167
C1–C5, C12–C15, C17, C18	11	100 nF 16 V X7R	0402	TDK C1005X7R1C104KT000F	C338032
C6, C9	2	10 µF 50 V X7S	1206	Samsung CL31Y106KBKVPNE	C2803389
C7, C8, C10, C11	4	22 µF 10 V X5R	0805	Samsung CL21A226KPCLRNC	C318688
C16	1	1 µF 10 V X7R	0402	Murata GRM155Z71A105KE01D	C528974
R1–R6, R9, R12, R13	9	10 kΩ 1 %	0402	Vishay CRCW040210K0FKED	C71617
R7, R8	2	4,7 kΩ 1 %	0402	Vishay RCC04024K70FKED	C1858949
R10, R11	2	5,1 kΩ 1 %	0402	Vishay CRCW04025K10FKTD	C4333224
R14, R15, R16	3	470 Ω 5 % (Versorgungs-LED, LED-Daten, Kettenausgang)	0402	Vishay CRCW0402470RJNED	C482183

Zusatzteile (nicht bestückt)

#	Teil	Beschreibung	LCSC
A1	Antennenkabel kinghelm KH-IPEX-SMA-W-150	u.FL (IPEX1) auf SMA-Einbaubuchse, 150 mm, mit Mutter	C2683343
A2	Antenne DreamLNK W10-2.4G	2,4 GHz, 3 dBi, SMA-Stecker, klappbar, ca. 105 mm	C381198
A3	Knopfzelle CR2032	Beliebige Marke; vor Ort kaufen (Lithiumzellen unterliegen Versandbeschränkungen)	–
A4	Gegenkabel für J4	XH-Gehäuse 2,5 mm, 6-polig, BOOMELE XH-6Y (C10389) mit Crimpkontakten XH-T (C5670), 6 × UL1007 24 AWG, 150 mm, anderes Ende abisoliert und verzinnt	Sonderkabel
A5	Gegenkabel für J3	XH-Gehäuse 2,5 mm, 14-polig, BOOMELE XH-14Y (C38729) mit Crimpkontakten XH-T (C5670), 14 × UL1007 24 AWG, 150 mm, anderes Ende abisoliert und verzinnt	Sonderkabel

- **Antenne:** die SMA-Einbaubuchse von A1 in der Gehäusewand befestigen (Bohrung ca. 6,5 mm), das u.FL-Ende auf U4 stecken und A2 aufschrauben. Normales SMA verwenden, nicht RP-SMA. Alternativen: Kabellängen 120 mm (C2683342) oder 200 mm (C2683345); Antenne BAT WIRELESS BW2.4JWX195-13KJ (195 mm, 8 dBi, C497018).
- **Bestellung:** A1 und A2 lassen sich bei LCSC bestellen und zusammen mit einem JLCPCB-Bestückungsauftrag versenden.
- **Gegenkabel A4/A5:** Fertige XH-Kabel mit 150-mm-Litzen sind kein Standard-Lagerartikel. Bestellen Sie sie über den Dienst „Custom Cables“ von LCSC (Mindestmenge 1, Angebot innerhalb von 24 Stunden): Gehäuse und Kontakte wie angegeben, Litze UL1007 24 AWG, Länge 150 mm, alle Positionen bestückt, anderes Ende abisoliert und verzinnt. Pin 1 des Gehäuses muss zu Pin 1 des Steckers passen. Sonderkabel lassen sich mit der LCSC/JLCPCB-Sendung zusammenlegen. Fertige Alternative nur für J4 (200 mm): BXCONN XH6P-24AWG-200-A1T (C53413416).
- **CR2032:** mit „+“ nach oben einsetzen.
- Eine microSD-Karte ist nicht enthalten.

8.5 Leiterplatte

Das Layout der Platinenrevision 1 wurde am 3. November 2026 fertiggestellt; die Platine trägt den Aufdruck *EBCTester V1.0* und den Datumscode *20261103*. Die Fertigungsdaten für JLCPCB liegen im Hardware-Repository unter `KiCad/jlcpcb/production_files`.

Größe	80 × 80 mm
Lagen	2 Kupferlagen, Platinendicke 1,6 mm, 35 µm Kupfer
Bestückung	Alle Bauteile auf der Oberseite: 53 SMD-Teile und 4 bedrahtete Teile (J3, J4, J5, BT1)
Kupferflächen	Durchgehende GND-Fläche auf der Unterseite; Flächen für +19 V, den 5-V-Wandlerausgang, +5 V und GND auf der Oberseite
Befestigungslöcher	H1–H4, 3,2 mm für M3-Schrauben, durchkontaktiert und mit GND verbunden
Fertigungsdateien	`GERBER-EBCTester.zip` (Gerber- und Bohrdaten), `BOM-EBCTester.csv` (Stückliste), `CPL-EBCTester.csv` (Bestückungsdatei)

Befestigungslöcher

Lage der Lochmitten, gemessen von der linken oberen Ecke der Platine (Ausrichtung wie im Layoutbild, J3/J4 rechts):

Loch	x (mm)	y (mm)	Lage
H1	64,0	5,0	oben rechts, links neben J3
H2	76,0	74,5	rechte untere Ecke
H3	5,5	25,0	linker Rand, unter dem microSD-Halter
H4	20,5	73,5	unten, rechts neben dem Taster RESET

Da die Befestigungslöcher mit GND verbunden sind, liegt ein Metallgehäuse auf Masse der Platine. Die Löcher bilden kein Rechteck – entnehmen Sie die Maße für ein Gehäuse dieser Tabelle oder der Layoutdatei.

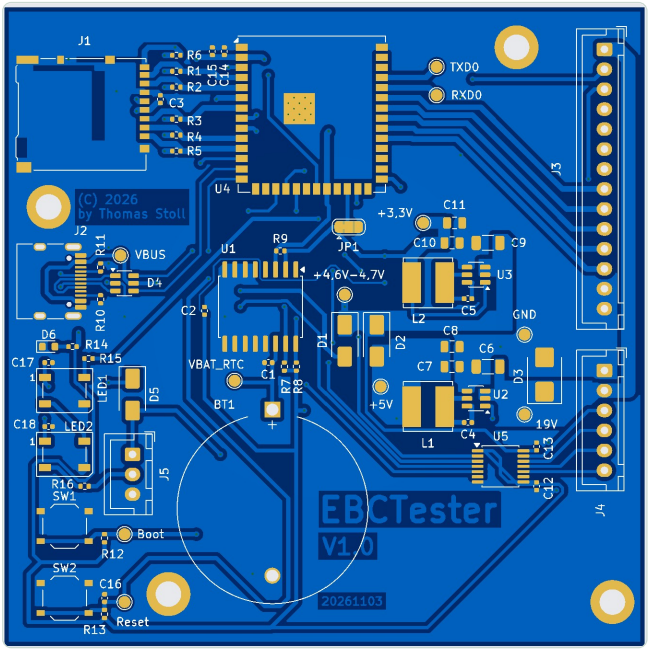


Abbildung 27: Layout, Oberseite (Kupfer und Bestückungsdruck)

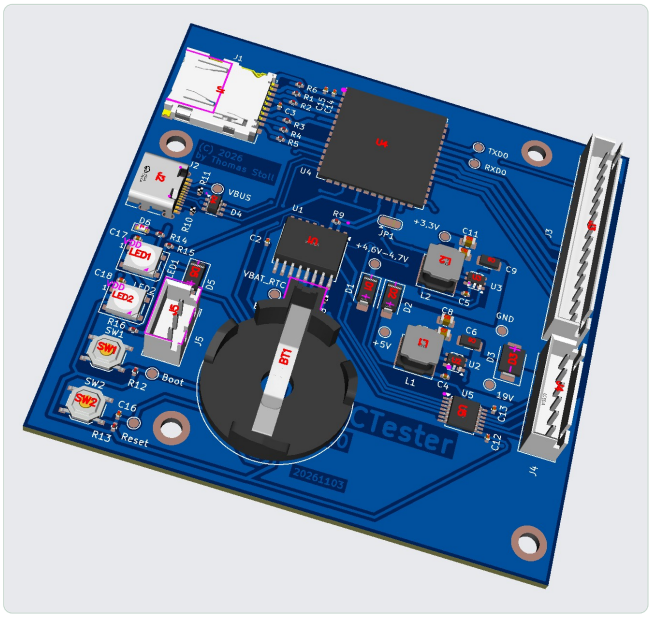


Abbildung 28: 3D-Ansicht der bestückten Platine

9 Firmware: Installation und Konfiguration

9.1 Release installieren (ohne Entwicklungsumgebung)

Jedes GitHub-Release enthält eine Datei `EBCTester-<version>-full-install.bin` mit Bootloader, Partitionstabelle und Anwendung. Sie wird an Adresse `0x0` geschrieben, zum Beispiel mit Espressifs esptool:

```
esptool.py --chip esp32s3 --port COM5 write_flash 0x0 EBCTester-1.4.72-full-install.bin
```

Wird die Platine nicht erkannt, im Download-Modus starten (BOOT halten, RESET drücken). Spätere Updates lassen sich über WLAN installieren (Abschnitt 4.4). Das Modul muss **16 MB Flash** haben.

> **Achtung**
> Release-Images verwenden die Standard-Pinbelegung der Firmware (Abschnitt 9.3). Prüfen Sie, ob sie zu Ihrer Hardware passt, bevor Sie ein Release-Image auf der EBCTester-Platine verwenden.

9.2 Aus dem Quellcode bauen

Voraussetzungen: VS Code mit der ESP-IDF-Erweiterung, **ESP-IDF 6.0.1**, Git und für den ersten Build eine Internetverbindung (verwaltete Komponenten und der SQLite-Quellcode werden heruntergeladen und per Hash geprüft).

```
git clone --recurse-submodules https://github.com/Gemelon/EBCTester.git
# vorhandener Checkout:
git submodule update --init --recursive
```

1. Das Projekt in VS Code öffnen, die ESP-IDF-Umgebung und das Ziel **esp32s3** wählen.
2. Platine, Flash, PSRAM, SD-Modus und UART-Pins in **menuconfig** prüfen (siehe unten).
3. Mit **ESP-IDF: Build your project** bauen.
4. Bei der Erstinstallation Bootloader, Partitionstabelle und Anwendung über USB flashen.

Ein ZIP-Download des Repositorys enthält die Submodule nicht. Führen Sie `git submodule update --remote` nicht aus, ohne die Änderungen zu prüfen – das Hauptrepository legt die getesteten Versionen fest. `sdkconfig.defaults` liefert portable Voreinstellungen; eine vorhandene lokale `sdkconfig` hat Vorrang. Größenoptimierung (`-Os`) ist voreingestellt, weil die Xtensa-Toolchain GCC 15.2 mit `-Og` in einem ESP-IDF-Treiber abstürzt.

Wichtige menuconfig-Optionen

Menü	Option	Standard
EBCTester → EBC battery tester UARTs	Tester-UART TX-/RX-GPIO, Baudrate	17 / 18, 9600
	Dongle-UART TX-/RX-GPIO, Baudrate	8 / 9, 9600
EBCTester → test control	Mittelungslänge (Meldungen je gespeichertem Messwert)	20
EBCTester → test store	Geräte-ID, max. ausstehende Messwerte	EBC-A20, 3000
EBCTester → Home Assistant MQTT	MQTT-Komponente einbauen	aus

Menü	Option	Standard
SD Card Manager	Busmodus (4-Bit-SDMMC / SPI), GPIOs, Takt	SDMMC, 10 MHz
	SDMMC CLK / CMD / D0 / D1 / D2 / D3	14 / 15 / 2 / 4 / 5 / 6
	GPIO der Kartenerkennung (aktiv high)	7
OTA Suite network	Passwort des Zugangspunkts (8–63 Zeichen), SNTP-Server	leer = zufällig, pool.ntp.org
OTA Suite web interface	Anfängliches Web-Anmeldepasswort	ebctester
OTA Suite update checks	URL des Firmware-Manifests	Platzhalter
OTA Suite firmware updates	Unverschlüsseltes HTTP-OTA erlauben	aus
	Anmeldepasswort für Einstellungsänderungen verlangen	ein
OTA Suite status LED	WS2812-LED einschalten, GPIO	ein, 48

9.3 Firmware-Voreinstellungen und Platinenrevision 1

Die Voreinstellungen der Firmware wurden für einen Entwicklungsaufbau mit einem ESP32-S3-DevKitC-1 gewählt. Die EBCTester-Platine Revision 1 verwendet eine andere Pinbelegung. Vor dem Einsatz der Platine müssen folgende Einstellungen übereinstimmen:

Signal	Platine Rev. 1	Firmware-Standard	Maßnahme
UART1 TX / RX (Tester)	GPIO 10 / 11	GPIO 17 / 18	in menuconfig einstellen
UART2 TX / RX (Dongle)	GPIO 8 / 9	GPIO 8 / 9	passt
SD CLK / CMD	GPIO 6 / 7	GPIO 14 / 15	in menuconfig einstellen
SD D0 / D1 / D2 / D3	GPIO 5 / 4 / 18 / 17	GPIO 2 / 4 / 5 / 6	in menuconfig einstellen
SD-Kartenerkennung	GPIO 13, **low** = Karte steckt	GPIO 7, **high** = Karte steckt	GPIO in menuconfig; die Polarität braucht eine Firmware-Option
Status-LED	WS2812B-Kette an GPIO 48 (LED1 Status, LED2 RX)	eine WS2812 an GPIO 48, nur Update-Animation	passt elektrisch; Status-/RX-Funktion geplant
Echtzeituhr DS3231	I²C GPIO 15/16, INT GPIO 14	nicht genutzt	künftige Erweiterung

Warnung
Der Kartenerkennungseingang von Firmware 1.4.72 kennt nur „aktiv high“; mit dem low-aktiven Schalter der Platine würde die Karte als fehlend gemeldet. Das muss vor dem Einsatz der Platine gelöst werden (Firmware-Option oder Hardwareänderung).

9.4 Partitionstabelle

Name	Typ	Offset	Größe	Verwendung
nvs	data/nvs	0x9000	24 KB	Einstellungen (WLAN, Passwörter, Modell, Plan, Batterie …)
otadata	data/ota	0xF000	8 KB	Aktiver Firmware-Slot
phy_init	data/phy	0x11000	4 KB	HF-Kalibrierung
ota_0 / ota_1	app	0x20000 / 0x590000	2 × 5,4 MB	Zwei Firmware-Slots für sichere Updates

Name	Typ	Offset	Größe	Verwendung
storage	data	0xB00000	5 MB	Reserviert, derzeit ungenutzt

Eine geänderte Partitionstabelle lässt sich nicht per OTA installieren; sie muss über USB geflasht werden. Sichern Sie vor einer Neupartitionierung Einstellungen und Daten.

9.5 Status-LED in Firmware 1.4.72

Firmware 1.4.72 steuert eine WS2812-RGB-LED (auf der EBCTester-Platine: LED1; beim ESP32-S3-DevKitC-1: GPIO 48 oder 38) und zeigt den Update-Zustand an: langsamer Farbwechsel = Normalbetrieb, blaues Pulsieren = Firmware wird geladen, gelbes Pulsieren = Prüfung, dauerhaft grün = wartet auf Neustart, rotes Blinken = Update fehlgeschlagen. Die Helligkeit ist auf 24/255 begrenzt. LED2 wird noch nicht angesteuert. Die in Abschnitt 2.5 beschriebenen Status- und RX-Funktionen ersetzen den Farbwechsel in einer kommenden Firmware-Version.

10 Kommunikation mit dem Tester

10.1 Serielle Schnittstelle

UART1 ist mit **9600 Baud, 8 Datenbits, ungerader Parität, 1 Stoppbit (8O1)** bei 5-V-TTL-Pegel mit dem Tester verbunden. Rahmen beginnen mit `0xFA` und enden mit `0xF8`. Statusrahmen sind 19 Byte lang. Vor jeder Verwendung der Daten wird eine XOR-Prüfsumme über die Bytes 1–16 geprüft. Der EBC-A20 überträgt Prüfsummen im Bereich 0xF0–0xFF mit gelöschtem oberem Nibble, um die Rahmenmarken zu vermeiden; auch diese Variante wird akzeptiert.

Sollwerte werden als zwei Bytes zur Basis 240 übertragen (*Wert = High × 240 + Low*), mit einer Auflösung von 0,01 A bzw. 0,01 V. Der Bereich reicht für 50 A; die Auflösung von 10 mA ist an echter A50L-Hardware noch nicht geprüft. Der Entladebefehl enthält zusätzlich eine Zeitgrenze in ganzen Minuten.

10.2 Modellerkennung

Byte 16 jedes Statusrahmens enthält die Modellkennung (normiert als Rohwert modulo 100). Bei Statusbytes ab 100 enthalten die Bytes 10 und 11 die Firmware-Version des Testers (*B10 × 240 + B11*), die Varianten mit derselben Kennung unterscheidet.

Kennung (dez / hex)	Firmware-Version	Modell	Freigegeben
5 / 0x05	302 / andere	EBC-A05+ / EBC-A05	nein
6 / 0x06	310 / andere	EBC-A10+ / EBC-A10H	nein
9 / 0x09	311 / andere	EBC-A20+ / **EBC-A20**	nur A20
10 / 0x0A	311 / andere	**EBC-A50L** / **EBC-A40L**	ja
20 / 0x14	310 / andere	EBD-M10 / EBD-A10H	nein
25 / 0x19	310 / 315 / andere	EBD-M03+ / EBD-M05 / EBD-M03	nein
38 / 0x26	310 / 311 / andere	EBD-A100L / EBC-A100L / EBD-A100	nein

Solange die Firmware-Version eines Testers mit Kennung 10 noch unbekannt ist, wird er als „EBC-A40L/A50L (Version unknown)“ angezeigt, und es gelten die vorsichtigeren Grenzen des A40L. Die Modellprüfung läuft bei jedem gültigen Rahmen. Passt das Modell nicht zum gewählten, werden Verbindungs- und Messzustand gelöscht, ausstehende Befehle verworfen, Trennen gesendet und das automatische Wiederverbinden unterdrückt.

10.3 Betriebsarten

Code	Betriebsart	Code	Betriebsart	Code	Betriebsart
0x00	D-CC gestoppt	0x0A	D-CC aktiv	0x14	D-CC beendet
0x01	D-CP gestoppt	0x0B	D-CP aktiv	0x15	D-CP beendet
0x02	C-CV gestoppt	0x0C	C-CV aktiv	0x16	C-CV beendet

11 Datenspeicherung

11.1 SQLite-Datenbank auf der SD-Karte

Die Datenbankdatei ist `/sdcard/ebc_a20.db`. Sie wird angelegt, wenn eine Karte eingebunden wird; startet das Gerät ohne Karte, wird sie angelegt, sobald eine Karte eingesteckt wird. Die Karte wird nie formatiert, und eine vorhandene Datenbank wird nie gelöscht. Eine Datei ohne gültigen SQLite-Header wird unter dem Namen `ebc_a20.db.invalid-XXXXXXXX` aufbewahrt, bevor eine neue Datenbank entsteht.

Tabelle	Inhalt
`ebc_battery`	Batterien (Bezeichnung, Hersteller, Seriennummer, Datum, Nennkapazität, Notizen)
`ebc_test_plan` / `ebc_test_plan_step`	Gespeicherte Testpläne und ihre Schritte
`ebc_test_run`	Testläufe (Batterie, Plan, Start/Ende, Status, aktueller Schritt, Zyklus, Firmware, Notizen, Anzeigename)
`ebc_run_segment`	Segmente eines Laufs (Betriebsart, Schritt, Zyklus, Sollwerte)
`ebc_measurement`	Messwerte (Zeit UTC und Laufzeit, Strom, Spannung, Kapazität …)
`ebc_resistance_result`	Drei DCIR-Ergebnisse je Test (Zeiten, U1/I1, U2/I2, mΩ, Gültigkeit)
`ebc_schema_metadata`, `ebc_sync_state`	Schemaversion; Fortschritt der MariaDB-Synchronisation

Die Views `ebc_measurement_view` und `ebc_resistance_result_view` erleichtern die Auswertung. SQLite läuft mit `synchronous=FULL`, echtem `fsync()`, Fremdschlüsseln und einer Integritätsprüfung beim Start.

11.2 Schemaversionen

Schema	Firmware	Änderung
1	–	Grundtabellen (entspricht den MariaDB-Migrationen 001–005)
2	DCIR-Unterstützung	Tabelle `ebc_resistance_result`
3	ab 1.4.67	Spalte `display_name` für eigene Laufnamen

Migrationen laufen beim ersten Start mit Karte automatisch in einer Transaktion. Ältere Firmware lehnt ein neueres Schema ab: Bewahren Sie vor einem Update eine Datenbanksicherung auf, falls Sie eventuell zurückwechseln müssen.

Zeitstempel werden in UTC gespeichert und setzen eine synchronisierte Uhr (SNTP) voraus. Ohne sie wird stattdessen die Laufzeit des Geräts gespeichert; eine Kalenderzeit wird nicht erfunden.

11.3 MariaDB-Sicherung

Der MariaDB-Synchronisierer kopiert Pläne, Schritte, Batterien, Läufe, Segmente, Messwerte und DCIR-Ergebnisse in ein bestehendes MariaDB-Schema (z. B. `ebc_a20`). Der Fortschritt wird je Lauf verfolgt; nur neue Daten werden in Blöcken von 200 Zeilen mit `INSERT IGNORE` gesendet.

- Das Grundschema (Migrationen 001–005) muss auf dem Server vorhanden sein. Das Repository enthält die Migration `sql/006_resistance_results_mariadb.sql` für DCIR-Ergebnisse, aber kein vollständiges Anfangsschema.

- Der Benutzer braucht ein Konto mit `mysql_native_password` und SELECT/INSERT/UPDATE auf `ebc_test_plan`, `ebc_battery`, `ebc_test_run`, zusätzlich DELETE auf `ebc_test_plan_step` und INSERT auf `ebc_run_segment` und `ebc_measurement`, außerdem Rechte auf die DCIR-Ergebnistabelle.
- Die Verbindung ist einfaches TCP ohne TLS.

Beispielabfrage der DCIR-Ergebnisse:

```
SELECT battery_label, test_started_at_utc, requested_seconds,
       actual_seconds, resistance_mohm, valid, error
FROM ebc_resistance_result_view
ORDER BY test_started_at_utc DESC, run_id, result_number;
```

12 Schnittstellen

12.1 HTTP-API

Die Weboberfläche nutzt eine JSON-API, die sich auch von eigenen Skripten verwenden lässt. Bei eingeschalteter Browser-Anmeldung verlangen alle Endpunkte eine Anmeldesitzung (Cookie); POST-Anfragen brauchen zusätzlich das CSRF-Token aus `GET /api/status` im Header `X-CSRF-Token`. Das Speichern von Einstellungen verlangt außerdem `X-Admin-Password`, wenn die Passwortbestätigung eingeschaltet ist.

Endpunkt	Methode	Zweck
`/api/auth/login, /api/auth/logout`	POST	Anmelden (JSON Benutzername/Passwort, Header `X-Web-Login: 1`) und Abmelden
`/api/status`	GET	Geräte-, Netzwerk- und Update-Status, CSRF-Token, `sessionDays`, `loginEnabled`
`/api/auth/settings`	POST	Anmeldeeinstellungen: `sessionDays` (1/7), `loginEnabled`, optional neues `password`
`/api/tester/status`	GET	Live-Werte, Verbindung, Modell und Grenzen, Plan, Aufzeichnung
`/api/tester/settings, /api/tester/catalog`	GET	Manuelle Werte, Batterie, Plan; gespeicherte Batterien und Pläne
`/api/tester/command`	POST	connect, disconnect, stop, auto_connect, charge, discharge_cc, discharge_cp
`/api/tester/plan, /api/tester/plan/action`	POST	Plan speichern; start, abort, reset, load_db, save_db
`/api/tester/battery, /api/tester/model`	POST	Batterieaktionen; Auswahl des Testermodells
`/api/tester/resistance(/history)`	GET/POST	DCIR-Konfiguration, Status, Start/Abbruch; Verlauf
`/api/db/runs, /api/db/run?id=`	GET	Batterien und die letzten 300 Läufe; Laufdetails und Segmente
`/api/db/measurements?id=&max=`	GET	Reihe `[segment, uptimeMs, epochMs, I, U, Ah]`, reduziert auf max Punkte (200–6000)
`/api/test-runs(/rename|/delete)`	GET/POST	Testlaufverwaltung
`/api/mariadb/…, /api/mqtt/…, /api/debug/…`	GET/POST	Status und Einstellungen der optionalen Dienste
`/api/sd/…, /api/updates/check, /api/ota`	GET/POST	Dateioperationen auf der SD-Karte; Update-Prüfung und -Installation

12.2 Home Assistant MQTT

Die MQTT-Komponente muss beim Bauen eingeschaltet werden (menuconfig) und wird in der Weboberfläche konfiguriert. Beim Verbinden veröffentlicht das Gerät ein dauerhaftes (retained) Discovery-Dokument unter `homeassistant/device/ebctester_<MAC>/config`, die Verfügbarkeit (`online` / `offline` als Last Will) unter `<prefix>/availability` und jeden aufgezeichneten Messwert (Spannung, Strom, Kapazität) als nicht dauerhafte QoS-0-Nachricht unter `<prefix>/state`. Entitäten für Betriebsart, Planstatus und Batterie werden bereits angekündigt, erhalten aber noch keine Werte.

Befehle werden unter `<prefix>/command` als JSON-Objekt angenommen:

```
{"command":"start_plan"}        (nur wenn Fernstart erlaubt ist)
{"command":"stop"}
{"command":"abort"}
{"command":"select_plan","name":"Planname"}
{"command":"select_battery","name":"Batteriebezeichnung"}
```

Über MQTT lassen sich weder freie Sollwerte setzen noch Datenbankeinträge anlegen. Ungültige Befehle werden ignoriert. Fehlgeschlagene Verbindungen werden mit wachsenden Abständen von 1 bis 60 Sekunden wiederholt.

13 Firmware-Updates und Veröffentlichung

13.1 Ablauf eines Updates

- Das Gerät lädt nur das Manifest `firmware.json` (nach dem WLAN-Verbinden, alle sechs Stunden und beim Öffnen der Oberfläche). HTTPS-Prüfungen warten auf eine synchronisierte Uhr.
- Versionen müssen rein numerisch im Format `major.minor.patch` und neuer als die installierte sein.
- Manifest und Image müssen direkt ausgeliefert werden (HTTP 200, keine Weiterleitungen). Bei HTTPS werden Zertifikatskette, Hostname und Gültigkeitsdaten geprüft. HTTP ist nur mit einer ausdrücklichen Build-Option für ein vertrauenswürdiges Testnetz möglich.
- Das Image wird in den inaktiven Slot gestreamt; exakte Größe und SHA-256 werden geprüft, bevor er zum Booten ausgewählt wird. Projektname und Zielplattform müssen passen.
- Nach dem Neustart muss die neue Firmware 10 Sekunden laufen; ein früherer Reset oder Absturz führt zurück zur vorherigen Version.

13.2 Format des Manifests

```
{
  "project": "EBCTester",
  "target": "esp32s3",
  "version": "1.4.73",
  "url": "https://updates.example.org/BatteryTester/EBCTester-1.4.73.bin",
  "size": 2734352,
  "sha256": "<64 Hexziffern in Kleinbuchstaben>",
  "notes": "Versionshinweise"
}
```

Alle sieben Felder sind Pflicht; das Manifest muss kleiner als 2048 Byte sein, die Hinweise höchstens 512 Byte. Die Image-URL muss im selben Verzeichnis wie das Manifest liegen und `<project>-<version>.bin` heißen.

13.3 Eigene Builds veröffentlichen

In einem frischen Checkout ist das Veröffentlichen abgeschaltet. Eingeschaltet wird es mit einer lokalen, von Git ignorierten Datei `CMakeUserConfig.cmake` (siehe README). Ein erfolgreicher Build erzeugt dann das unveränderliche versionierte Image, eine Kompatibilitätskopie und zuletzt das Manifest, wahlweise per SFTP hochgeladen. Eine veröffentlichte Version wird nie mit anderem Inhalt überschrieben – erhöhen Sie zuerst die Version. Geben Sie das Lizenzverzeichnis immer zusammen mit der Firmware weiter.

14 Sicherheit und Einschränkungen

14.1 Sicherheit

- **Anmeldung:** Benutzer `admin` mit dem Web-Anmeldepasswort (anfangs `ebctester`, festgelegt durch `CONFIG_OTA_SUITE_WEB_PASSWORD`; ein in den Anmeldeeinstellungen gespeichertes Passwort liegt im NVS-Namensraum `web_auth` und hat Vorrang). Die Browser-Anmeldung lässt sich abschalten; dann sind Status, Daten und Bedienung ohne Sitzung zugänglich. Sitzungen verwenden ein zufälliges 256-Bit-Cookie (`HttpOnly`, `SameSite=Lax`), gelten genau 1 oder 7 Tage und liegen im RAM (ein Neustart beendet sie). Höchstens 8 Sitzungen gleichzeitig; fehlgeschlagene Anmeldungen werden um eine Sekunde verzögert.
- **CSRF:** Alle ändernden Anfragen verlangen ein CSRF-Token; Einstellungen verlangen zusätzlich das aktuelle Anmeldepasswort im Header `X-Admin-Password`, unabhängig davon, ob die Browser-Anmeldung eingeschaltet ist (beim Bauen einstellbar). Änderungen an Passwort oder Anmelderegel machen alle Sitzungen ungültig.
- **Übertragung:** Weboberfläche, MQTT und MariaDB sind unverschlüsselt. Nur in vertrauenswürdigen Netzen verwenden und Port 80 nie ins Internet weiterleiten.
- **Speicher:** NVS und Flash sind standardmäßig nicht verschlüsselt; schützen Sie den physischen Zugang und den USB-Anschluss (das Startprotokoll zeigt das Passwort des Zugangspunkts). Ändern Sie das für alle Geräte gleiche Anfangspasswort `ebctester` beim ersten Gebrauch.
- **Firmware:** SHA-256 schützt vor Beschädigung, nicht vor absichtlichem Austausch. Secure Boot, signierte Images und Flash-Verschlüsselung sind eigene Entscheidungen beim Einsatz.

14.2 Bekannte Einschränkungen (Firmware 1.4.72)

- Die UART2-Brücke für die PC-Software des Herstellers ist nicht umgesetzt.
- Die Echtzeituhr DS3231 wird noch nicht genutzt; Datum und Uhrzeit kommen nur per SNTP.
- Nur EBC-A20, EBC-A40L und EBC-A50L lassen sich steuern; Praxistests mit dem A50L stehen noch aus.
- Keine Profile für Zellchemien und keine Temperaturmessung.
- MQTT veröffentlicht nur Messwerte; Statusentitäten erhalten noch keine Werte.
- Platinenrevision 1 erfordert Anpassungen der Firmware (Abschnitt 9.3) und ist noch nicht aufgebaut und getestet.
- Die Status- und RX-Funktionen von LED1/LED2 sind noch nicht umgesetzt.
- Ein vergessenes Anmeldepasswort lässt sich nur durch Löschen des Flash-Speichers zurücksetzen.

14.3 Lizenz

Copyright 2026 Thomas Stoll. EBCTester, OTA Suite und SD Card Manager stehen unter der Apache License 2.0 – kommerzielle Nutzung, Änderung und Verkauf sind nach ihren Bedingungen erlaubt; ihre Gewährleistungs- und Haftungsausschlüsse gelten. Komponenten Dritter behalten ihre eigenen Lizenzen: Bootstrap 5.3.3 und Chart.js 4.4.9 (MIT), SQLite (gemeinfrei), ESP-IDF (Apache 2.0) und weitere, aufgeführt in `THIRD_PARTY_NOTICES.md`. EBCTester ist ein unabhängiges Projekt und kein offizielles Produkt von ZKETECH; Produktnamen sind Marken ihrer jeweiligen Inhaber.

Anhang A – Sprachdateien

Weitere Sprachen der Benutzeroberfläche werden aus dem Ordner `/languages` der SD-Karte geladen. Eine vollständige Vorlage ist `sd_card/languages/en-custom.json` im Repository; ein unvollständiges Beispiel für Filipino (Tagalog) ist `fil.json`.

1. Die Vorlage kopieren, z. B. als `fr.json`.
2. `"code": "fr"` und `"name": "Français"` setzen.
3. Nur die Werte in `"strings"` übersetzen; alle Schlüssel unverändert lassen.
4. Platzhalter wie `{p0}`, `%u`, `%s` beibehalten (printf-Platzhalter in derselben Reihenfolge).
5. Als UTF-8 ohne BOM speichern, nach `/languages/fr.json` hochladen (z. B. im Reiter SD-Karte), auf **Sprachen neu laden** klicken und die Sprache in der Kopfzeile wählen.

```
{
  "version": 1,
  "code": "fr",
  "name": "Français",
  "strings": {
    "language.title": "Langue",
    "...": "..."
  }
}
```

Fehlende Schlüssel fallen auf Englisch zurück. Dateien mit dem Namen `en.json` oder `de.json` ersetzen die eingebauten Texte, solange sie vorhanden sind. Grenzen: max. 16 Dateien, 128 KB je Datei, 1024 Einträge, 4096 Byte je Wert, 80 Byte für den Namen. Sprachcodes bestehen aus Kleinbuchstaben, Ziffern und Bindestrichen (max. 24 Zeichen), z. B. `fr`, `en-gb`.

Die eingebaute deutsche Übersetzung von Firmware 1.4.72 schreibt Umlaute noch als ae/oe/ue und lässt einige Karten englisch. Mit einer eigenen `de.json` auf der SD-Karte lässt sich das bereits jetzt korrigieren.

Anhang B – Glossar

AP	Access Point (Zugangspunkt) – das eigene WLAN des Geräts für Einrichtung und Rückfallzugang.
C-CV	Laden mit konstantem Strom, danach mit konstanter Spannung (CC/CV).
D-CC / D-CP	Entladen mit konstantem Strom / konstanter Leistung.
DCIR	Gleichstrom-Innenwiderstand (Direct Current Internal Resistance), gemessen mit einem Lastpuls.
CSRF	Cross-Site Request Forgery; das Token schützt vor Anfragen fremder Webseiten.
Rahmen	Eine Meldung zwischen EBCTester und Tester (engl. frame).
NVS	Non-Volatile Storage – nichtflüchtiger Einstellungsspeicher im Flash des ESP32.
OTA	Over the Air – Firmware-Update über WLAN.
Segment	Abschnitt eines Testlaufs mit einer Betriebsart.
SNTP	Simple Network Time Protocol – Zeitabgleich über das Internet.
SOC	Ladezustand (State of Charge) in Prozent.
Testlauf	Aufzeichnung eines manuellen Vorgangs, Plans oder DCIR-Tests mit allen Messwerten.